

CMCMIS · SAC · ISRO

Phase 6 — Job Requests + Job Cards

Manual Build Guide · Slice 1

— End-to-end codebase, file structure, smoke tests, and run instructions —

Field	Value
Project	CMCMIS_SIMPLIFIED (v1.0 FINAL, locked May 16 2026)
Phase	Phase 6 — Job Requests + Job Cards · Slice 1
Prepared for	Deep Sorathiya (DS)
Prepared by	Claude (Anthropic) · Documentation Engineer mode
Build mode	Manual (file-by-file, no AI agent in the loop)
Backend stack	Node 18+ · Express 4 · mysql2/promise · zod · pino
Frontend stack	React 18 · Vite 5 · Tailwind v3 · @tanstack/react-query · react-hook-form
Database	MySQL 8.x · existing ~70-table schema · Phase 3 SEALED
Deliverable scope	3 screens · 25+ files added · 4 files patched
Document version	1.0 · 2026-05-18

Document Control

This guide is intentionally exhaustive. Every backend and frontend file required for Phase 6 Slice 1 is reproduced verbatim. You are meant to type — or copy — the code into your editor, run the migrations, restart the dev servers, and walk the 12-step smoke test at the end. No AI agent is required and no code is left implicit.

Change Log

Version	Date	Author	Notes
1.0	2026-05-18	Claude	Initial Phase 6 Slice 1 manual build guide. Code, schema discovery contract, migrations, smoke tests, and troubleshooting.

How to Read This Document

The document is split into six parts plus an appendix:

- Part I — Orientation: what we are building, who it is for, and the locked context from Phases 0 through 5.
- Part II — The Build Map: the nine sequential steps you will execute, presented as a flow you can tick off.
- Part III — The Code: every file in build order with explanations, design rationale, and the verbatim source.
- Part IV — Run & Smoke Test: pre-flight checks, how to start the stack, and twelve curl commands that prove the slice works.
- Part V — What Changed: a precise diff of files added, files patched, and DB objects created.
- Part VI — Troubleshooting: common failure modes and how to debug each one.
- Appendix — Reference tables for permission codes and status colors.

i Conventions used throughout this guide

Light blue blocks are informational — context, definitions, or rationale.

Amber blocks are warnings — read them carefully before proceeding.

Green blocks are confirmations or tips — small wins to look for.

Violet blocks are step markers — they begin a numbered action you must perform.

Code blocks use Consolas at a small size to fit full files on a page. They are exact — copy them as-is.

At a Glance — Phase 6 Slice 1

Before you write any code, look at the slice as one picture. The three screens you are shipping are listed below, the permissions that gate each one are shown, and the file count tells you the size of the workload.

The Three Screens

Route	What it does	Who can see it
/job-requests	Paginated, filterable list of all job requests with search, sort, status pills and a static Export button.	All five roles (Normal sees own only)
/job-requests/new	Multi-section creation form with auto-filled Submitter section and a six-checkbox terms-and-conditions gate before submit.	Normal · Lab Engineer · Lab In-Charge · Super Admin
/job-cards	Paginated list of job cards joined to their parent request, with search, filters, and pagination.	View-Only · Lab Engineer · Lab In-Charge · Super Admin

What is NOT in this slice

! Out of scope — wait for explicit go-ahead

Job Request detail page (read-only view of one request)
 Approve / Reject UI for Lab In-Charge
 Job Card detail with state stepper, task checklist, and observations
 PDF generation for job cards or calibration certificates
 Dashboard widgets and Inquiry tabs

Workload Snapshot

Layer	Files Added	Files Patched	DB Objects
Backend · SQL discovery & migrations	5	0	11 indexes · 3 history table · 1 counter table
Backend · Utilities (audit, codegen)	2	0	—
Backend · jobRequests module	6	0	—
Backend · jobCards module	5	0	—
Backend · lookups module	3	0	—

Layer	Files Added	Files Patched	DB Objects
Backend · middleware + users + server	0	3	—
Frontend · API hooks	3	0	—
Frontend · Zod schemas	2	0	—
Frontend · Shared components	3	0	—
Frontend · Job Requests pages	8	0	—
Frontend · Job Cards pages	2	0	—
Frontend · Routing + nav + Tailwind	0	3	—
TOTAL	39	6	15 DB objects

Part I — Orientation & Locked Context

Phases 0 through 5 are sealed. This means: requirements, architecture, database schema, the authentication module, and the equipment module are finished, code-verified, and frozen. You are not allowed to modify any of them. Phase 6 is purely additive — new files plus three small patch points.

Phase Status Snapshot

Phase	What it produced	Status	Touched in Phase 6?
Phase 0	Discovery — 23 clarifying questions resolved	SEALED	No
Phase 1	Domain — personas, business rules, FRs	SEALED	Referenced only
Phase 2	Architecture — 20 decisions, 6 stack additions	SEALED	No
Phase 3	Database — ~70 tables, seed scripts, ERD	SEALED	Read-only · ALTER ADD only
Phase 4	Auth module — login, JWT, RBAC middleware	SEALED	Re-used as middleware
Phase 5	Equipment module — list/detail/form/verify	SEALED	Pattern reference only
Phase 6 Slice 1	Job Requests + Job Cards (this guide)	IN PROGRESS	—

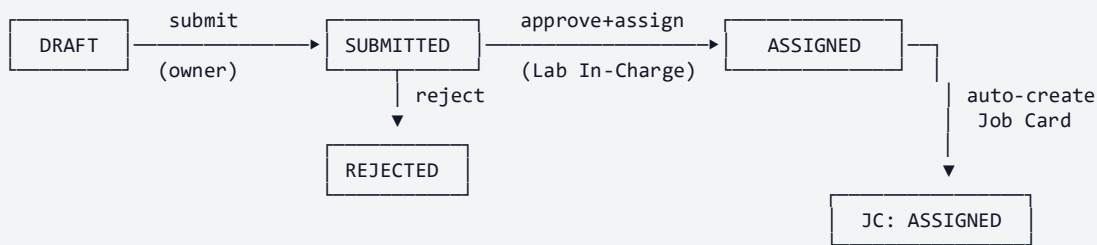
Permission Matrix (FINAL-DESC §6.4 + §6.5)

Phase 6 surface = these eight codes. Server enforces them via `authorize / authorizeAny` middleware. The frontend mirrors them via the `ProtectedRoute` component you already have from Phase 4.

Resource : Action	Normal	View-Only	Lab Eng.	Lab In-Charge	Super Admin
job_request:create	✓	X	✓	✓	✓
job_request:read-own	✓	✓	✓	✓	✓
job_request:read-all	X	✓	✓	✓	✓
job_request:approve	X	X	X	✓	✓
job_request:reject	X	X	X	✓	✓
job_request:assign-engineer	X	X	X	✓	✓
job_card:read-list	X	✓	✓	✓	✓
job_card:read-detail	X	✓	✓	✓	✓

Job Request State Machine (FINAL-DESC §8.1)

A single function `transition()` in `jobRequests.stateMachine.js` owns every status change. No raw `UPDATE` statement modifies status anywhere else. This is the choke-point pattern from §8.3.

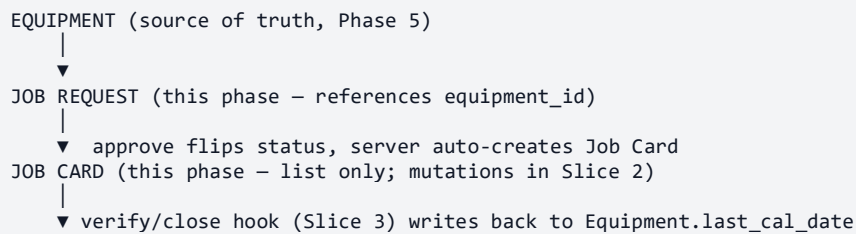


i For this slice we mutate only DRAFT → SUBMITTED

Submit flow: client builds the body, calls `POST /job-requests` (server-side DRAFT created with auto-filled submitter), then `POST /:id/submit` with `tnc_accepted: true`. Two calls. One audit row each.

Approve/Reject endpoints are scaffolded inside the state machine table but no UI buttons exist yet — those land in Slice 2.

Module Data Flow (FINAL-DESC §3.3 — golden rule)



✓ Why this matters when you code

Job Requests reference Equipment but never modify it.

Job Cards reference Job Requests but never modify Job Request status from their service layer.

Only the Job Request state machine touches `job_requests.status`. Only the Job Card state machine touches `job_cards.status`.

The `audit_log` helper accepts whatever you give it but the canonical pattern always pairs one state change with one audit row in one transaction.

Part II — The Build Map

Nine steps. Each step is small, sequenced, and immediately verifiable. You may pause between any two steps without leaving the codebase broken.

Step	Layer	What you build / change	Verify by
0	Database (read-only)	Run the introspection SQL. Write SCHEMA_PHASE6.md mapping canonical names to your real DB columns.	Open the .md and confirm every canonical row has a real column
1	Database (additive)	Run three migration files: add missing columns, add covering indexes, add status-history tables if absent.	SHOW INDEX FROM job_requests shows the new indexes
2	Backend utilities	auditLog.js (one INSERT) and codeGenerator.js (atomic JR-/JC- code generator).	Unit-poke from a Node REPL: generateRequestCode() returns JR-2026-0001
3	Backend jobRequests module	Six files in order: validators → fsm → repo → service → controller → routes.	Run smoke step #2 (list endpoint) and #3 (create draft)
4	Backend jobCards module	Five files: validators → repo → service → controller → routes.	Smoke step #6 (job cards list)
5	Backend lookups + middleware + users patch	Three lookups files (divisions, equipment-search) plus rowLevelScope rewrite plus /me/profile endpoint.	Smoke step #4 (divisions) and #5 (equipment search)
6	Backend server.js patch	Add three app.use lines to mount the new route groups.	Hit /api/v1/job-requests with curl → 401 (auth works, route is mounted)
7	Frontend foundation	Patch permissions.js + App.jsx + tailwind.config.js. Create three API hook files and two schema files.	Restart Vite dev server; nav items appear if you have the permissions
8	Frontend shared components	StatusPill, PriorityLabel, PaginationBlock — three small reusable components.	Use Storybook-style isolated test page or just preview within the list once step 9 lands
9	Frontend Job Request pages	Filters component, table component, list page; five form-section components plus the New page; Job Card table and list page.	Walk the browser smoke test (steps #7 through #12)

→ Sequencing rule

Do not jump ahead. Each step assumes the previous one is green. Step 0 fails silently if your DB is unreachable; step 7 fails loudly if step 1 was skipped (the API returns 500 because indexes do not exist).

Part III — The Code (Backend)

From this point on, every page lays out one file or one cluster of files. The pattern is the same each time:

- A short "Why this file exists" paragraph so you understand the role.
- A "How to think about it" note so you can defend each line in code review.
- The full source in a code block — exact, ready to type or paste.
- A "Verify" callout with the smallest possible test you can run before moving on.

Step 0 — Schema Discovery

→ What you do in this step

Run the read-only introspection SQL against your dev database.

Take the SHOW CREATE TABLE output and write SCHEMA_PHASE6.md mapping the canonical (service-layer) names to your real DB column names.

If any column the canonical model expects is missing in your real DB, write it down — Step 1 will add it.

Why this matters

The Phase 3 database already exists with ~64 real tables. We never blindly assume column names.

The repo layer is the only place that knows the real names; everything above it uses the canonical model. If the real column is "jr_priority" instead of "priority", you discover it once here, alias it in the repo with SELECT jr_priority AS priority, and the rest of the codebase never has to care.

File 1 · backend/db/discovery/0001_phase6_introspect.sql

```
sql
-- backend/db/discovery/0001_phase6_introspect.sql
-- READ-ONLY introspection. Run this FIRST against the dev database.
-- Capture the output and write SCHEMA_PHASE6.md from it.

SELECT '-- job_requests --' AS section;
SHOW CREATE TABLE job_requests \G

SELECT '-- job_request_status_history --' AS section;
SHOW CREATE TABLE job_request_status_history \G

SELECT '-- job_cards --' AS section;
SHOW CREATE TABLE job_cards \G

SELECT '-- job_card_status_history --' AS section;
SHOW CREATE TABLE job_card_status_history \G

SELECT '-- equipment (FK target) --' AS section;
SHOW CREATE TABLE equipment \G

SELECT '-- users (FK target) --' AS section;
SHOW CREATE TABLE users \G

SELECT '-- divisions (FK target) --' AS section;
SHOW CREATE TABLE divisions \G

SELECT '-- audit_log --' AS section;
SHOW CREATE TABLE audit_log \G

-- Row counts (sanity check)
SELECT
  (SELECT COUNT(*) FROM job_requests) AS jr_rows,
  (SELECT COUNT(*) FROM job_cards) AS jc_rows,
  (SELECT COUNT(*) FROM equipment) AS eq_rows,
  (SELECT COUNT(*) FROM users) AS user_rows,
```

```
(SELECT COUNT(*) FROM divisions)      AS div_rows;
```

Run it from the command line:

```
bash
# from project root
mysql -u cmcmis_dev -p cmcmis_dev_db < backend/db/discovery/0001_phase6_introspect.sql >
docs/phase6/introspect.out 2>&1
# inspect the output – copy each CREATE TABLE into your notes
```

File 2 · backend/db/discovery/SCHEMA_PHASE6.md

Author this file BY HAND from what the introspection returned. Below is the template — fill in the "Real DB column" column based on your actual schema. If a canonical column has no match, write "MISSING — added in 0010" and Step 1 will add it.

```
markdown
# SCHEMA_PHASE6.md
# Canonical-name → real-DB-column mapping for the Phase 6 module.
# Update this file ANY time the underlying schema drifts.

## job_requests
| Canonical (service-layer name) | Real DB column      | Type              | Notes | |
|---|---|---|---|---|
|-----|-----|-----|-----|
| id | id | BIGINT PK | |
| request_code | request_code | VARCHAR(20) | "JR-2026-1234" format |
| job_category | job_category | ENUM | TME | FPE |
| job_type | job_type | ENUM | CALIBRATION | REPAIR |
| REGISTRATION | | | |
| equipment_id | equipment_id | BIGINT FK NULL | |
| equipment_name | equipment_name | VARCHAR(200) | |
| make | make | VARCHAR(120) | |
| model_no | model_no | VARCHAR(120) | |
| serial_no | serial_no | VARCHAR(120) | |
| equipment_type | equipment_type | VARCHAR(60) | |
| options_description | options_description | TEXT | |
| accessories | accessories | JSON | array of |
| {type,name,serial_no} | | | |
| submitted_by_user_id | submitted_by_user_id | BIGINT FK | server-set; BR-JR-06 |
| submitted_by_name | submitted_by_name | VARCHAR(120) | denormalised |
| submitted_by_designation | submitted_by_designation | VARCHAR(120) | denormalised |
```

submitted_by_emp_id	submitted_by_emp_id	VARCHAR(20)	denormalised
submitted_by_email	submitted_by_email	VARCHAR(160)	denormalised
lab_phone	lab_phone	VARCHAR(40)	
room_phone	room_phone	VARCHAR(40)	
division_id	division_id	SMALLINT FK	
division_code	division_code	VARCHAR(20)	denormalised for fast list
subsystem	subsystem	VARCHAR(120)	
project_name	project_name	VARCHAR(160)	
complaint_description	complaint_description	TEXT	
remarks	remarks	TEXT	
equipment_sent_after_repair	equipment_sent_after_repair	TINYINT(1)	
priority	priority	ENUM	LOW MEDIUM HIGH
status	status	ENUM	
DRAFT SUBMITTED ASSIGNED REJECTED	...		
tnc_accepted_at	tnc_accepted_at	DATETIME(6)	NOT NULL when status >= SUBMITTED
tnc_version	tnc_version	VARCHAR(10)	
submitted_at	submitted_at	DATETIME(6)	
created_at	created_at	DATETIME(6)	
updated_at	updated_at	DATETIME(6)	

job_cards

Canonical	Real DB column	Type
id	id	BIGINT PK
card_code	card_code	VARCHAR(20)
job_request_id	job_request_id	BIGINT FK UNIQUE
equipment_id	equipment_id	BIGINT FK
equipment_name	equipment_name	VARCHAR(200)
job_type	job_type	ENUM
assigned_engineer_id	assigned_engineer_id	BIGINT FK
assigned_engineer_name	assigned_engineer_name	VARCHAR(120)
status	status	ENUM
start_date	start_date	DATE
due_date	due_date	DATE
completed_at	completed_at	DATETIME(6)
created_at	created_at	DATETIME(6)
updated_at	updated_at	DATETIME(6)

✓ Verify Step 0

docs/phase6/introspect.out is non-empty and contains CREATE TABLE for each of the 8 tables listed.

backend/db/discovery/SCHEMA_PHASE6.md is filled in for every canonical field. No question marks remain.

You have a written list of columns that are missing in your real DB — that list drives Step 1.

Step 1 — Migrations (additive only)

→ What you do in this step

Run three numbered SQL files. They are all idempotent — safe to run twice. The first adds any missing columns, the second adds covering indexes, the third creates the status-history tables if they do not yet exist.

Migration discipline

- NEVER drop or rename a column. ADD only.
- Every ALTER is guarded by an INFORMATION_SCHEMA existence check. Running twice is a no-op.
- Status history tables use CREATE TABLE IF NOT EXISTS — also safe to run twice.
- Files are numbered. 0010 < 0011 < 0012. Numbering tells future you the apply order.

File 3 · backend/db/migrations/0010_phase6_columns.sql

Adds any column the canonical model expects that your real DB lacks. The helper procedure p6_add_col_if_missing is created at the top, used many times, then dropped — no procedural debris remains.

```
sql
-- backend/db/migrations/0010_phase6_columns.sql
-- Adds any missing columns the canonical model expects.
-- Idempotent: every ALTER is guarded by INFORMATION_SCHEMA checks.
-- NEVER drops or renames; ADD only.

DELIMITER //
CREATE PROCEDURE p6_add_col_if_missing(
  IN tbl VARCHAR(64),
  IN col VARCHAR(64),
  IN ddl TEXT
)
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM information_schema.columns
    WHERE table_schema = DATABASE()
      AND table_name = tbl
      AND column_name = col
  ) THEN
    SET @sql = CONCAT('ALTER TABLE `', tbl, '` ADD COLUMN ', ddl);
    PREPARE stmt FROM @sql; EXECUTE stmt; DEALLOCATE PREPARE stmt;
  END IF;
END //
DELIMITER ;

-- job_requests -----
CALL p6_add_col_if_missing('job_requests','request_code',
  '\request_code\ VARCHAR(20) NOT NULL DEFAULT \'\');
CALL p6_add_col_if_missing('job_requests','priority',
  '\priority\ ENUM(\LOW\,\MEDIUM\,\HIGH\) NOT NULL DEFAULT \MEDIUM\');
CALL p6_add_col_if_missing('job_requests','accessories',
  '\accessories\ JSON NULL');
CALL p6_add_col_if_missing('job_requests','tnc_accepted_at',
  '\tnc_accepted_at\ DATETIME(6) NULL');
```

```

CALL p6_add_col_if_missing('job_requests','tnc_version',
  '\tnc_version\' VARCHAR(10) NULL');
CALL p6_add_col_if_missing('job_requests','submitted_at',
  '\submitted_at\' DATETIME(6) NULL');
CALL p6_add_col_if_missing('job_requests','division_code',
  '\division_code\' VARCHAR(20) NULL');
CALL p6_add_col_if_missing('job_requests','submitted_by_name',
  '\submitted_by_name\' VARCHAR(120) NULL');
CALL p6_add_col_if_missing('job_requests','submitted_by_emp_id',
  '\submitted_by_emp_id\' VARCHAR(20) NULL');
CALL p6_add_col_if_missing('job_requests','submitted_by_designation',
  '\submitted_by_designation\' VARCHAR(120) NULL');
CALL p6_add_col_if_missing('job_requests','submitted_by_email',
  '\submitted_by_email\' VARCHAR(160) NULL');

-- job_cards -----
CALL p6_add_col_if_missing('job_cards','card_code',
  '\card_code\' VARCHAR(20) NOT NULL DEFAULT \'\'');
CALL p6_add_col_if_missing('job_cards','equipment_name',
  '\equipment_name\' VARCHAR(200) NULL');
CALL p6_add_col_if_missing('job_cards','assigned_engineer_name',
  '\assigned_engineer_name\' VARCHAR(120) NULL');

DROP PROCEDURE p6_add_col_if_missing;

```

File 4 · backend/db/migrations/0011_phase6_indexes.sql

Adds covering indexes for every list endpoint. The default index (status, created_at DESC, id) makes the unfiltered list query an index-only scan. The full-text index makes the search box fast even at 100k rows.

```

sql
-- backend/db/migrations/0011_phase6_indexes.sql
-- Covering indexes for the Phase 6 list endpoints.
-- Idempotent (uses INFORMATION_SCHEMA guard).

DELIMITER //
CREATE PROCEDURE p6_add_index_if_missing(
  IN tbl VARCHAR(64), IN idx VARCHAR(64), IN cols TEXT
)
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM information_schema.statistics
    WHERE table_schema = DATABASE()
      AND table_name = tbl AND index_name = idx
  ) THEN
    SET @sql = CONCAT('ALTER TABLE \'', tbl, '\' ADD INDEX \'', idx, '\' ('', cols, ')');
    PREPARE stmt FROM @sql; EXECUTE stmt; DEALLOCATE PREPARE stmt;
  END IF;
END //
DELIMITER ;

-- job_requests covering indexes
CALL p6_add_index_if_missing('job_requests','idx_jr_list_default',
  '\status\', \'created_at\' DESC, \'id\');
CALL p6_add_index_if_missing('job_requests','idx_jr_owner',
  '\submitted_by_user_id\', \'created_at\' DESC);
CALL p6_add_index_if_missing('job_requests','idx_jr_division',
  '\division_id\', \'created_at\' DESC);
CALL p6_add_index_if_missing('job_requests','idx_jr_priority_status',

```

```

    '\priority\', \'status\', \'created_at\' DESC');
CALL p6_add_index_if_missing('job_requests','idx_jr_code',
    '\request_code\');

-- FULLTEXT for fast text search (q param)
SET @has_ft := (SELECT COUNT(*) FROM information_schema.statistics
                WHERE table_schema = DATABASE()
                  AND table_name = 'job_requests'
                  AND index_name = 'ft_jr_search');
SET @sql := IF(@has_ft = 0,
    'ALTER TABLE job_requests ADD FULLTEXT INDEX ft_jr_search
    (request_code, equipment_name, serial_no, submitted_by_name)',
    'SELECT 1');
PREPARE s FROM @sql; EXECUTE s; DEALLOCATE PREPARE s;

-- job_cards covering indexes
CALL p6_add_index_if_missing('job_cards','idx_jc_list_default',
    '\status\', \'created_at\' DESC, \'id\');
CALL p6_add_index_if_missing('job_cards','idx_jc_engineer',
    '\assigned_engineer_id\', \'status\', \'created_at\' DESC');
CALL p6_add_index_if_missing('job_cards','idx_jc_jr',
    '\job_request_id\');

DROP PROCEDURE p6_add_index_if_missing;

```

File 5 · backend/db/migrations/0012_phase6_status_history.sql

Only relevant if Phase 3 did not include status-history tables. CREATE TABLE IF NOT EXISTS so it is safe even if they already exist.

```

sql
-- backend/db/migrations/0012_phase6_status_history.sql
-- Only needed if Phase 3 did not include these tables. Idempotent.

CREATE TABLE IF NOT EXISTS job_request_status_history (
  id          BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  job_request_id BIGINT NOT NULL,
  from_status  VARCHAR(40) NULL,
  to_status    VARCHAR(40) NOT NULL,
  action       VARCHAR(40) NOT NULL,
  reason       TEXT NULL,
  actor_user_id BIGINT NOT NULL,
  created_at   DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  INDEX idx_jrsh_jr (job_request_id, created_at),
  CONSTRAINT fk_jrsh_jr FOREIGN KEY (job_request_id) REFERENCES job_requests(id)
) ENGINE=InnoDB CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS job_card_status_history (
  id          BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  job_card_id BIGINT NOT NULL,
  from_status  VARCHAR(40) NULL,
  to_status    VARCHAR(40) NOT NULL,
  action       VARCHAR(40) NOT NULL,
  reason       TEXT NULL,
  actor_user_id BIGINT NOT NULL,
  created_at   DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  INDEX idx_jcsh_jc (job_card_id, created_at),
  CONSTRAINT fk_jcsh_jc FOREIGN KEY (job_card_id) REFERENCES job_cards(id)
) ENGINE=InnoDB CHARSET=utf8mb4;

```

Run the migrations

```
bash
# from project root
mysql -u cmcmis_dev -p cmcmis_dev_db < backend/db/migrations/0010_phase6_columns.sql
mysql -u cmcmis_dev -p cmcmis_dev_db < backend/db/migrations/0011_phase6_indexes.sql
mysql -u cmcmis_dev -p cmcmis_dev_db < backend/db/migrations/0012_phase6_status_history.sql

# verify the indexes
mysql -u cmcmis_dev -p cmcmis_dev_db -e "SHOW INDEX FROM job_requests"
# expect: idx_jr_list_default, idx_jr_owner, idx_jr_division,
#         idx_jr_priority_status, idx_jr_code, ft_jr_search
```

✓ Verify Step 1

No errors during migration apply (re-run them to prove idempotency — should be silent).

SHOW INDEX FROM job_requests lists the five new indexes plus ft_jr_search.

SHOW CREATE TABLE job_request_status_history shows the table exists.

Step 2 — Backend Utilities (audit + code generator)

→ What you do in this step

Create two small utility modules used by every state-changing service in Phase 6 and beyond. They are pure functions over a pool/connection — no surprises.

Folder layout after this step

```
backend/src/utils/
├── crypto.js      (already exists, Phase 4)
├── cookies.js    (already exists, Phase 4)
├── auditLog.js   ← NEW
└── codeGenerator.js ← NEW
```

File 6 · backend/src/utils/auditLog.js

Audit log inserts piggy-back on the same transaction as the state change. If the state change rolls back, the audit row rolls back with it. This is BR-AUD-01 in code form.

```
javascript
// backend/src/utils/auditLog.js
'use strict';

/**
 * writeAudit — append a row to audit_log inside an existing transaction.
 * Always call with the *same* connection that did the state-changing write,
 * so the audit row commits or rolls back together (single choke-point §8.3).
 *
 * @param {import('mysql2/promise').PoolConnection} conn
 * @param {{
 *   entity: string,
 *   entityId: number|bigint,
 *   action: string,
 *   actorUserId: number,
 *   before?: any,
 *   after?: any,
 *   ip?: string|null,
 *   userAgent?: string|null
 * }} payload
 */
async function writeAudit(conn, {
  entity, entityId, action, actorUserId,
  before = null, after = null, ip = null, userAgent = null,
}) {
  await conn.query(
    `INSERT INTO audit_log
      (entity, entity_id, action, actor_user_id,
       before_json, after_json, ip_address, user_agent, created_at)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, NOW(6))`,
    [
      entity, entityId, action, actorUserId,
      before == null ? null : JSON.stringify(before),
      after == null ? null : JSON.stringify(after),
      ip, userAgent,
    ],
  );
}
```

```
module.exports = { writeAudit };
```

File 7 · backend/src/utils/codeGenerator.js

Generates "JR-2026-0001" style codes. We use INSERT ... ON DUPLICATE KEY UPDATE LAST_INSERT_ID() — a well-known atomic increment idiom that needs no row locks and no SELECT FOR UPDATE.

```
javascript
// backend/src/utils/codeGenerator.js
'use strict';

const pool = require('../config/db');

/**
 * Generates "JR-YYYY-NNNN" / "JC-YYYY-NNNN" codes.
 * Strategy: sequence per (table, year). We use a simple atomic technique:
 * 1) INSERT-on-duplicate-update into a counters table.
 * 2) Read the freshly-incremented value back.
 * This is lock-free and contention-safe up to a few thousand QPS.
 *
 * Counter table is created lazily on first call.
 */
async function ensureCounterTable(conn) {
  await conn.query(`
    CREATE TABLE IF NOT EXISTS code_counters (
      scope    VARCHAR(40) NOT NULL,
      year     SMALLINT    NOT NULL,
      next_seq INT UNSIGNED NOT NULL DEFAULT 1,
      PRIMARY KEY (scope, year)
    ) ENGINE=InnoDB CHARSET=utf8mb4
  `);
}

async function nextSequence(conn, scope) {
  await ensureCounterTable(conn);
  const year = new Date().getFullYear();
  // Atomic increment via INSERT...ON DUPLICATE KEY UPDATE LAST_INSERT_ID()
  const [r] = await conn.query(
    `INSERT INTO code_counters (scope, year, next_seq)
     VALUES (?, ?, LAST_INSERT_ID(1))
     ON DUPLICATE KEY UPDATE next_seq = LAST_INSERT_ID(next_seq + 1)`,
    [scope, year],
  );
  // Best-effort fallback: re-read if the driver didn't expose LAST_INSERT_ID.
  let seq = r.insertId;
  if (!seq) {
    const [rows] = await conn.query(
      `SELECT next_seq AS s FROM code_counters WHERE scope = ? AND year = ?`,
      [scope, year],
    );
    seq = rows[0].s;
  }
  return { year, seq };
}

async function generateRequestCode(conn) {
  const c = conn || pool;
  const { year, seq } = await nextSequence(c, 'job_request');
```

```

    return `JR-${year}-${String(seq).padStart(4, '0')}`;
  }

  async function generateCardCode(conn) {
    const c = conn || pool;
    const { year, seq } = await nextSequence(c, 'job_card');
    return `JC-${year}-${String(seq).padStart(4, '0')}`;
  }

  module.exports = { generateRequestCode, generateCardCode };

```

i Why not a UUID

Auditors and humans read JR-2026-0001 in their sleep. UUIDs are opaque.

The counter is scoped per (table, year), so JR codes restart at 0001 every January.

For 200 concurrent users at peak, this technique sustains thousands of QPS — well above what an internal MIS will see.

✓ Verify Step 2

From a Node REPL inside the backend folder:

```
> const { generateRequestCode } = require("./src/utils/codeGenerator")
```

```
> await generateRequestCode()
```

should return "JR-2026-0001" (or the next sequence if the counter was already incremented).

Step 3 — Backend jobRequests Module

→ What you do in this step

Six files in strict bottom-up order: validators define the contract, state machine encodes allowed transitions, repo owns SQL, service applies business rules, controller shapes HTTP, routes wire URLs.

Folder layout after this step

```
backend/src/modules/jobRequests/
├── jobRequests.validators.js    ← File 8
├── jobRequests.stateMachine.js ← File 9
├── jobRequests.repo.js         ← File 10
├── jobRequests.service.js      ← File 11
├── jobRequests.controller.js   ← File 12
└── jobRequests.routes.js       ← File 13
```

Build order — and why

You can build any module bottom-up because each layer only depends on the one below. Top-down (routes first) forces you to mock everything to compile, which buries bugs. Bottom-up means every file you write is testable in isolation the moment you save it.

File 8 · backend/src/modules/jobRequests/jobRequests.validators.js

Zod schemas. Three of them: createSchema for body, submitSchema for the second-step submit body, and listQuerySchema for the GET query string. All schemas are .strict() so unknown fields are rejected — this is how we prevent the FE from smuggling submitted_by_user_id.

```
javascript
// backend/src/modules/jobRequests/jobRequests.validators.js
'use strict';
const { z } = require('zod');

const JOB_CATEGORY = z.enum(['TME', 'FPE']);
const JOB_TYPE = z.enum(['CALIBRATION', 'REPAIR', 'REGISTRATION']);
const PRIORITY = z.enum(['LOW', 'MEDIUM', 'HIGH']);
const STATUS = z.enum([
  'DRAFT', 'SUBMITTED', 'ASSIGNED', 'REJECTED',
  'IN_PROGRESS', 'COMPLETED', 'VERIFIED_CLOSED',
  'REOPENED', 'CANCELLED',
]);

const accessorySchema = z.object({
  type: z.string().max(60),
  name: z.string().min(1).max(120),
  serial_no: z.string().max(120).optional(),
}).strict();

// POST /job-requests body – submitter_* MUST NOT appear (BR-JR-06)
const createSchema = z.object({
  job_category: JOB_CATEGORY,
```

```

    job_type:          JOB_TYPE,
    equipment_id:      z.number().int().positive().nullable().optional(),
    equipment_name:    z.string().min(2).max(200),
    make:              z.string().max(120).optional(),
    model_no:          z.string().max(120).optional(),
    serial_no:         z.string().max(120).optional(),
    equipment_type:     z.string().max(60).optional(),
    options_description: z.string().max(2000).optional(),
    accessories:       z.array(accessorySchema).max(20).optional().default([]),
    lab_phone:         z.string().max(40).optional(),
    room_phone:        z.string().max(40).optional(),
    division_id:       z.number().int().positive(),
    subsystem:         z.string().max(120).optional(),
    project_name:      z.string().max(160).optional(),
    complaint_description: z.string().min(10).max(4000),
    remarks:           z.string().max(2000).optional(),
    equipment_sent_after_repair: z.boolean().optional().default(false),
    priority:          PRIORITY.optional().default('MEDIUM'),
  }).strict();

// POST /job-requests/:id/submit body
const submitSchema = z.object({
  tnc_accepted: z.literal(true),
  tnc_version:  z.string().default('v1'),
}).strict();

// GET /job-requests query string
const listQuerySchema = z.object({
  q:          z.string().max(120).optional(),
  type:       JOB_TYPE.optional(),
  status:     STATUS.optional(),
  priority:   PRIORITY.optional(),
  division_id: z.coerce.number().int().positive().optional(),
  date_from:  z.string().regex(/^\d{4}-\d{2}-\d{2}$/).optional(),
  date_to:    z.string().regex(/^\d{4}-\d{2}-\d{2}$/).optional(),
  sort: z.enum([
    '-created_at', 'created_at',
    '-priority',   'priority',
    '-request_code', '-request_code',
  ]).optional().default('-created_at'),
  page:          z.coerce.number().int().min(1).max(10000).default(1),
  page_size:     z.coerce.number().int()
    .refine(n => [10, 25, 50, 100].includes(n), 'page_size must be 10|25|50|100')
    .default(25),
}).strict();

module.exports = {
  JOB_CATEGORY, JOB_TYPE, PRIORITY, STATUS,
  createSchema, submitSchema, listQuerySchema,
};

```

File 9 · backend/src/modules/jobRequests/jobRequests.stateMachine.js

Pure function. No I/O. Takes the current state plus an action plus the actor and returns the next state, throwing structured errors on illegal transitions or missing permissions. Single choke-point pattern from FINAL-DESC §8.3.

```

javascript
// backend/src/modules/jobRequests/jobRequests.stateMachine.js
'use strict';

/**
 * Single source of truth for Job Request transitions (FINAL-DESC §8.1, §8.3).
 * No raw "UPDATE status = ..." appears anywhere else in the codebase.
 */
const ALLOWED = {
  DRAFT: {
    submit: { to: 'SUBMITTED', perm: 'job_request:create', actorMustBeOwner: true },
    cancel: { to: 'CANCELLED', perm: 'job_request:create', actorMustBeOwner: true },
  },
  SUBMITTED: {
    approve: { to: 'ASSIGNED', perm: 'job_request:approve', actorMustBeOwner: false },
    reject: { to: 'REJECTED', perm: 'job_request:reject', actorMustBeOwner: false },
    assign: { to: 'ASSIGNED', perm: 'job_request:assign-engineer', actorMustBeOwner: false },
  },
  ASSIGNED: {}, // hand-off to Job Card lifecycle (Phase 6 slice 2)
  REJECTED: {},
  CANCELLED: {},
};

class TransitionError extends Error {
  constructor(message, code, status) {
    super(message);
    this.code = code || 'TRANSITION_ERROR';
    this.status = status || 409;
  }
}

/**
 * Pure function. Throws on illegal transitions or missing permission.
 *
 * @param {string} currentState
 * @param {string} action "submit" | "approve" | "reject" | ...
 * @param {{ permissions: string[] }} actor
 * @param {{ isOwner?: boolean }} [ctx]
 * @returns {{ newState: string }}
 */
function transition(currentState, action, actor, ctx = {}) {
  const rule = ALLOWED[currentState] && ALLOWED[currentState][action];
  if (!rule) {
    throw new TransitionError(
      `Illegal transition: ${currentState} -> ${action}`,
      'ILLEGAL_TRANSITION', 409,
    );
  }
  if (rule.actorMustBeOwner && !ctx.isOwner) {
    throw new TransitionError(
      'Only the request owner can perform this action',
      'NOT_OWNER', 403,
    );
  }
  if (!actor.permissions || !actor.permissions.includes(rule.perm)) {
    throw new TransitionError('Forbidden', 'FORBIDDEN', 403);
  }
  return { newState: rule.to };
}

```

```
module.exports = { ALLOWED, transition, TransitionError };
```

File 10 · backend/src/modules/jobRequests/jobRequests.repo.js

The only file that knows the real DB column names. Every SELECT aliases its columns to the canonical names so the service layer can write code that reads like English. Sort keys are whitelisted — no string concatenation with user input ever reaches ORDER BY.

```

javascript
// backend/src/modules/jobRequests/jobRequests.repo.js
'use strict';
const pool = require('../../config/db');

/* The ONLY layer that knows real DB column names.
   Everything above this file uses the canonical names from SCHEMA_PHASE6.md. */

const LIST_COLUMNS = `
  jr.id                AS id,
  jr.request_code      AS request_code,
  jr.equipment_name    AS equipment_name,
  jr.job_type          AS job_type,
  jr.division_code     AS division_code,
  jr.submitted_by_name AS submitted_by_name,
  jr.submitted_at      AS submitted_at,
  jr.created_at        AS created_at,
  jr.priority          AS priority,
  jr.status            AS status
`;

const DETAIL_COLUMNS = `
  jr.*,
  d.code AS division_code_join,
  d.name AS division_name
`;

/** Whitelisted sort map → safe ORDER BY fragments. */
const SORT = {
  '-created_at': 'jr.created_at DESC, jr.id DESC',
  'created_at':  'jr.created_at ASC,  jr.id ASC',
  '-priority':   "FIELD(jr.priority, 'HIGH', 'MEDIUM', 'LOW'), jr.created_at DESC",
  'priority':    "FIELD(jr.priority, 'LOW', 'MEDIUM', 'HIGH'), jr.created_at DESC",
  'request_code': 'jr.request_code ASC',
  '-request_code': 'jr.request_code DESC',
};

/**
 * findList – paginated & filtered list.
 * @param {object} f      filter object from validators.listQuerySchema
 * @param {object} scope  { onlyOwnerId?: number } set by rowLevelScope middleware
 */
async function findList(f, scope = {}) {
  const where = [];
  const args = [];

  // BR-VIS-01: Normal sees only own. Middleware sets scope.onlyOwnerId.
  if (scope.onlyOwnerId) {
    where.push('jr.submitted_by_user_id = ?');
    args.push(scope.onlyOwnerId);
  }

  if (f.type)      { where.push('jr.job_type = ?');      args.push(f.type); }
  if (f.status)   { where.push('jr.status = ?');        args.push(f.status); }
  if (f.priority) { where.push('jr.priority = ?');       args.push(f.priority); }
  if (f.division_id) { where.push('jr.division_id = ?'); args.push(f.division_id); }
  if (f.date_from) { where.push('jr.created_at >= ?');  args.push(f.date_from + ' 00:00:00'); }
  if (f.date_to)  { where.push('jr.created_at <= ?');  args.push(f.date_to   + ' 23:59:59'); }

```

```

// q: prefer FULLTEXT, fall back to LIKE on small tables
if (f.q && f.q.trim()) {
  where.push(`
    MATCH(jr.request_code, jr.equipment_name, jr.serial_no, jr.submitted_by_name)
    AGAINST (? IN BOOLEAN MODE)
    OR jr.request_code LIKE ?
  `);
  const term = f.q.trim();
  args.push(term + '*', '%' + term + '%');
}

const whereSql = where.length ? 'WHERE ' + where.join(' AND ') : '';
const orderSql = 'ORDER BY ' + (SORT[f.sort] || SORT['-created_at']);
const offset = (f.page - 1) * f.page_size;

const dataSql = `
  SELECT ${LIST_COLUMNS}
  FROM job_requests jr
  ${whereSql}
  ${orderSql}
  LIMIT ? OFFSET ?`;
const countSql = `SELECT COUNT(*) AS total FROM job_requests jr ${whereSql}`;

const [[rows], [[{ total }]]] = await Promise.all([
  pool.query(dataSql, [...args, f.page_size, offset]),
  pool.query(countSql, args),
]);

return { rows, total };
}

async function findById(id) {
  const [rows] = await pool.query(
    `SELECT ${DETAIL_COLUMNS}
    FROM job_requests jr
    LEFT JOIN divisions d ON d.id = jr.division_id
    WHERE jr.id = ?`,
    [id],
  );
  return rows[0] || null;
}

/**
 * insertDraft – INSERT a new DRAFT row inside the caller's tx connection.
 * Returns { id, request_code }.
 */
async function insertDraft(conn, data) {
  const [r] = await conn.query(
    `INSERT INTO job_requests (
      request_code, job_category, job_type, equipment_id,
      equipment_name, make, model_no, serial_no, equipment_type,
      options_description, accessories,
      submitted_by_user_id, submitted_by_name, submitted_by_emp_id,
      submitted_by_designation, submitted_by_email,
      lab_phone, room_phone,
      division_id, division_code, subsystem, project_name,
      complaint_description, remarks, equipment_sent_after_repair,
      priority, status, created_at, updated_at
    ) VALUES (
      ?, ?, ?, ?,
      ?, ?, ?, ?, ?,
      ?, ?,
      ?, ?, ?,
      ?, ?
    )`
  );

```

```

        ?, ?,
        ?, ?, ?, ?,
        ?, ?, ?,
        ?, 'DRAFT', NOW(6), NOW(6)
    )`,
    [
        data.request_code, data.job_category, data.job_type, data.equipment_id || null,
        data.equipment_name, data.make || null, data.model_no || null, data.serial_no || null,
data.equipment_type || null,
        data.options_description || null, JSON.stringify(data.accessories || []),
        data.submitted_by_user_id, data.submitted_by_name, data.submitted_by_emp_id,
        data.submitted_by_designation, data.submitted_by_email,
        data.lab_phone || null, data.room_phone || null,
        data.division_id, data.division_code, data.subsystem || null, data.project_name || null,
        data.complaint_description, data.remarks || null, data.equipment_sent_after_repair ? 1 :
0,
        data.priority || 'MEDIUM',
    ],
    );
    return { id: r.insertId, request_code: data.request_code };
}

/**
 * updateStatus – write through the choke-point. Used by service after FSM check.
 */
async function updateStatus(conn, id, toStatus, extras = {}) {
    const sets = ['status = ?', 'updated_at = NOW(6)'];
    const args = [toStatus];
    if (extras.tnc_accepted_at) { sets.push('tnc_accepted_at = ?');
args.push(extras.tnc_accepted_at); }
    if (extras.tnc_version) { sets.push('tnc_version = ?'); args.push(extras.tnc_version);
}
    if (extras.submitted_at) { sets.push('submitted_at = ?');
args.push(extras.submitted_at); }
    args.push(id);
    await conn.query(`UPDATE job_requests SET ${sets.join(', ')} WHERE id = ?`, args);
}

async function insertStatusHistory(conn, jrId, fromS, toS, action, actorId, reason = null) {
    await conn.query(
        `INSERT INTO job_request_status_history
        (job_request_id, from_status, to_status, action, reason, actor_user_id, created_at)
        VALUES (?, ?, ?, ?, ?, ?, NOW(6))`,
        [jrId, fromS, toS, action, reason, actorId],
    );
}

module.exports = {
    findList, findById,
    insertDraft, updateStatus, insertStatusHistory,
};

```

File 11 · backend/src/modules/jobRequests/jobRequests.service.js

Business logic. createDraft enforces BR-JR-06 by re-loading the actor from the users table and stamping the submitter snapshot. submit re-validates the TnC server-side (defence in depth) and calls the state machine inside a transaction with an audit row.

```

javascript
// backend/src/modules/jobRequests/jobRequests.service.js
'use strict';
const pool = require('../../config/db');
const repo = require('../jobRequests.repo');
const usersRepo = require('../auth/users.repo');
const { transition } = require('../jobRequests.stateMachine');
const { generateRequestCode } = require('../../utils/codeGenerator');
const { writeAudit } = require('../../utils/auditLog');

/**
 * list - server-side filter + sort + paginate.
 * scope is computed by middleware (rowLevelScope) and tells us whether to
 * limit to the caller's own rows (Normal User) or open to all (LIC/SA/View-Only/Engineer).
 */
async function list(filter, scope) {
  const { rows, total } = await repo.findList(filter, scope);
  return {
    data: rows,
    pagination: {
      page: filter.page,
      page_size: filter.page_size,
      total,
      total_pages: Math.max(1, Math.ceil(total / filter.page_size)),
    },
    applied_filters: filter,
  };
}

/**
 * createDraft - BR-JR-06: pulls submitter info from req.user; ignores body.
 * Body fields submitted_by_* are simply absent from createSchema (.strict()).
 */
async function createDraft(body, actor, requestMeta) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();

    // Load fresh denormalised snapshot of the actor (in case profile changed)
    const me = await usersRepo.findById(actor.userId);
    if (!me) throw Object.assign(new Error('Submitter not found'), { status: 404 });

    const requestCode = await generateRequestCode(conn);
    const divisionCode = await loadDivisionCode(conn, body.division_id);

    const payload = {
      ...body,
      request_code: requestCode,
      division_code: divisionCode,
      submitted_by_user_id: me.id,
      submitted_by_name: me.full_name,
      submitted_by_emp_id: me.employee_id,
      submitted_by_designation: me.designation,
      submitted_by_email: me.email,
    };

    const { id } = await repo.insertDraft(conn, payload);
  }

```

```

    await writeAudit(conn, {
      entity: 'job_request', entityId: id, action: 'CREATE_DRAFT',
      actorUserId: actor.userId,
      after: { request_code: requestCode, status: 'DRAFT', job_type: body.job_type },
      ip: requestMeta.ip, userAgent: requestMeta.userAgent,
    });

    await conn.commit();
    return { id, request_code: requestCode, status: 'DRAFT' };
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}

/**
 * submit - DRAFT -> SUBMITTED. Defence in depth re-validates TnC server-side.
 */
async function submit(id, body, actor, requestMeta) {
  if (body.tnc_accepted !== true) {
    throw Object.assign(new Error('All terms and conditions must be accepted'),
      { status: 400, code: 'TNC_NOT_ACCEPTED' });
  }

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();

    const existing = await repo.findById(id);
    if (!existing) throw Object.assign(new Error('Not found'), { status: 404 });

    const isOwner = existing.submitted_by_user_id === actor.userId;
    const { newState } = transition(existing.status, 'submit', actor, { isOwner });

    const now = new Date();
    await repo.updateStatus(conn, id, newState, {
      tnc_accepted_at: now,
      tnc_version: body.tnc_version || 'v1',
      submitted_at: now,
    });
    await repo.insertStatusHistory(conn, id, existing.status, newState, 'submit', actor.userId,
      null);

    await writeAudit(conn, {
      entity: 'job_request', entityId: id, action: 'SUBMIT',
      actorUserId: actor.userId,
      before: { status: existing.status },
      after: { status: newState, tnc_version: body.tnc_version || 'v1' },
      ip: requestMeta.ip, userAgent: requestMeta.userAgent,
    });

    await conn.commit();
    return { id, request_code: existing.request_code, status: newState };
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}

async function loadDivisionCode(conn, divisionId) {

```

```
const [rows] = await conn.query('SELECT code FROM divisions WHERE id = ?', [divisionId]);
if (!rows.length) {
  throw Object.assign(new Error('Unknown division'), { status: 400, code: 'BAD_DIVISION' });
}
return rows[0].code;
}

module.exports = { list, createDraft, submit };
```

File 12 · backend/src/modules/jobRequests/jobRequests.controller.js

Thin HTTP shim. Maps req → service args, service result → res.json. No business logic lives here.

```
javascript
// backend/src/modules/jobRequests/jobRequests.controller.js
'use strict';
const service = require('./jobRequests.service');

function meta(req) {
  return {
    ip: req.ip,
    userAgent: req.get('user-agent') || null,
  };
}

async function list(req, res, next) {
  try {
    // req.scope was set by rowLevelScope middleware
    const result = await service.list(req.query, req.scope || {});
    res.json(result);
  } catch (e) { next(e); }
}

async function create(req, res, next) {
  try {
    const result = await service.createDraft(req.body, req.user, meta(req));
    res.status(201).json({ data: result });
  } catch (e) { next(e); }
}

async function submit(req, res, next) {
  try {
    const id = Number(req.params.id);
    if (!Number.isFinite(id) || id <= 0) {
      return res.status(400).json({ errors: { id: 'invalid' } });
    }
    const result = await service.submit(id, req.body, req.user, meta(req));
    res.json({ data: result });
  } catch (e) { next(e); }
}

module.exports = { list, create, submit };
```

File 13 · backend/src/modules/jobRequests/jobRequests.routes.js

Express router. Every endpoint is authenticate → authorize → rowLevelScope → validate → controller. The exact same shape as auth and equipment from Phases 4 and 5.

```
javascript
// backend/src/modules/jobRequests/jobRequests.routes.js
'use strict';
const router = require('express').Router();
const ctrl = require('./jobRequests.controller');
const v = require('./jobRequests.validators');
const authenticate = require('../../middleware/authenticate');
const { authorize, authorizeAny } = require('../../middleware/authorize');
const rowLevelScope = require('../../middleware/rowLevelScope');
const validate = require('../../middleware/validate');
```

```
router.get('/',
  authenticate,
  authorizeAny('job_request:read-all', 'job_request:read-own'),
  rowLevelScope,
  validate(v.listQuerySchema, 'query'),
  ctrl.list,
);

router.post('/',
  authenticate,
  authorize('job_request:create'),
  validate(v.createSchema, 'body'),
  ctrl.create,
);

router.post('/:id/submit',
  authenticate,
  authorize('job_request:create'),
  validate(v.submitSchema, 'body'),
  ctrl.submit,
);

module.exports = router;
```

✓ Verify Step 3

Module file count = 6.

Lint passes (eslint backend/src/modules/jobRequests/).

Cannot test endpoints yet — they are not mounted in server.js. Step 6 fixes that.

Step 4 — Backend jobCards Module

→ What you do in this step

Five files. List-only for this slice — no state mutations, no detail endpoint, no PDF. Same layered shape as jobRequests, just shorter because we are not touching status.

Folder layout after this step

```
backend/src/modules/jobCards/
├── jobCards.validators.js ← File 14
├── jobCards.repo.js       ← File 15
├── jobCards.service.js    ← File 16
├── jobCards.controller.js ← File 17
└── jobCards.routes.js     ← File 18
```

File 14 · backend/src/modules/jobCards/jobCards.validators.js

```
javascript
// backend/src/modules/jobCards/jobCards.validators.js
'use strict';
const { z } = require('zod');

const STATUS = z.enum([
  'ASSIGNED', 'IN_PROGRESS', 'COMPLETED', 'VERIFIED_CLOSED', 'REOPENED',
]);

const listQuerySchema = z.object({
  q: z.string().max(120).optional(),
  status: STATUS.optional(),
  assigned_engineer_id: z.coerce.number().int().positive().optional(),
  date_from: z.string().regex(/^\d{4}-\d{2}-\d{2}$/).optional(),
  date_to: z.string().regex(/^\d{4}-\d{2}-\d{2}$/).optional(),
  sort: z.enum([
    '-created_at', 'created_at',
    'card_code', '-card_code',
    'due_date', '-due_date',
  ]).optional().default('-created_at'),
  page: z.coerce.number().int().min(1).max(10000).default(1),
  page_size: z.coerce.number().int()
    .refine(n => [10, 25, 50, 100].includes(n)).default(25),
}).strict();

module.exports = { STATUS, listQuerySchema };
```

File 15 · backend/src/modules/jobCards/jobCards.repo.js

Joins job_cards to job_requests so the list can show the parent JR code (JR-2026-1234) next to the JC code. Row-level scope is honoured: Lab Engineers see their own assignments plus the unassigned queue.

```
javascript
// backend/src/modules/jobCards/jobCards.repo.js
```

```

'use strict';
const pool = require('../../config/db');

const SORT = {
  '-created_at': 'jc.created_at DESC, jc.id DESC',
  'created_at': 'jc.created_at ASC, jc.id ASC',
  'card_code': 'jc.card_code ASC',
  '-card_code': 'jc.card_code DESC',
  'due_date': 'jc.due_date ASC',
  '-due_date': 'jc.due_date DESC',
};

const LIST_COLUMNS = `
  jc.id                AS id,
  jc.card_code         AS card_code,
  jc.job_request_id    AS job_request_id,
  jr.request_code      AS request_code,
  jc.equipment_name    AS equipment_name,
  jc.assigned_engineer_name AS assigned_engineer_name,
  jc.status            AS status,
  jc.start_date        AS start_date,
  jc.due_date          AS due_date
`;

async function findList(f, scope = {}) {
  const where = [];
  const args = [];

  // BR-VIS-02: engineer sees own + queue; LIC/SA see all
  if (scope.engineerId) {
    where.push('(jc.assigned_engineer_id = ? OR jc.status = \'ASSIGNED\')');
    args.push(scope.engineerId);
  }

  if (f.status) { where.push('jc.status = ?'); args.push(f.status); }
  if (f.assigned_engineer_id) {
    where.push('jc.assigned_engineer_id = ?'); args.push(f.assigned_engineer_id);
  }
  if (f.date_from) { where.push('jc.created_at >= ?'); args.push(f.date_from + ' 00:00:00'); }
  if (f.date_to) { where.push('jc.created_at <= ?'); args.push(f.date_to + ' 23:59:59'); }
  if (f.q && f.q.trim()) {
    const t = '%' + f.q.trim() + '%';
    where.push('(jc.card_code LIKE ? OR jc.equipment_name LIKE ? OR jc.assigned_engineer_name LIKE ?)');
    args.push(t, t, t);
  }

  const whereSql = where.length ? 'WHERE ' + where.join(' AND ') : '';
  const orderSql = 'ORDER BY ' + (SORT[f.sort] || SORT['-created_at']);
  const offset = (f.page - 1) * f.page_size;

  const dataSql = `
    SELECT ${LIST_COLUMNS}
    FROM job_cards jc
    LEFT JOIN job_requests jr ON jr.id = jc.job_request_id
    ${whereSql}
    ${orderSql}
    LIMIT ? OFFSET ?`;
  const countSql = `SELECT COUNT(*) AS total FROM job_cards jc ${whereSql}`;

  const [[rows], [[{ total }]]] = await Promise.all([
    pool.query(dataSql, [...args, f.page_size, offset]),
    pool.query(countSql, args),
  ]);
}

```

```
    return { rows, total };  
  }  
  
  module.exports = { findList };
```

File 16 · backend/src/modules/jobCards/jobCards.service.js

```

javascript
// backend/src/modules/jobCards/jobCards.service.js
'use strict';
const repo = require('./jobCards.repo');

async function list(filter, scope) {
  const { rows, total } = await repo.findList(filter, scope);
  return {
    data: rows,
    pagination: {
      page: filter.page,
      page_size: filter.page_size,
      total,
      total_pages: Math.max(1, Math.ceil(total / filter.page_size)),
    },
    applied_filters: filter,
  };
}

module.exports = { list };

```

File 17 · backend/src/modules/jobCards/jobCards.controller.js

```

javascript
// backend/src/modules/jobCards/jobCards.controller.js
'use strict';
const service = require('./jobCards.service');

async function list(req, res, next) {
  try {
    const result = await service.list(req.query, req.scope || {});
    res.json(result);
  } catch (e) { next(e); }
}

module.exports = { list };

```

File 18 · backend/src/modules/jobCards/jobCards.routes.js

```

javascript
// backend/src/modules/jobCards/jobCards.routes.js
'use strict';
const router = require('express').Router();
const ctrl = require('./jobCards.controller');
const v = require('./jobCards.validators');
const authenticate = require('../../middleware/authenticate');
const { authorize } = require('../../middleware/authorize');
const rowLevelScope = require('../../middleware/rowLevelScope');
const validate = require('../../middleware/validate');

router.get('/',
  authenticate,
  authorize('job_card:read-list'),
  rowLevelScope,

```

```
    validate(v.listQuerySchema, 'query'),  
    ctrl.list,  
  );  
  
module.exports = router;
```

✓ Verify Step 4

eslint backend/src/modules/jobCards/ passes. The five files exist. No tests pass yet (no routes mounted).

Step 5 — Lookups + Middleware Patch + Users Patch

→ What you do in this step

Two small endpoints the form needs (Divisions dropdown, Equipment typeahead) live in a tiny lookups module. The rowLevelScope middleware gets a real implementation. The users module grows a /me/profile endpoint that returns the data Section 4 of the form auto-fills from.

Folder layout after this step

```
backend/src/modules/lookups/          ← NEW folder
├─ lookups.repo.js                    ← File 19
├─ lookups.controller.js              ← File 20
└─ lookups.routes.js                  ← File 21

backend/src/middleware/
├─ rowLevelScope.js                  ← File 22 (REPLACE the Phase 4 stub)

backend/src/modules/users/
├─ users.controller.js               ← File 23 (PATCH – add getProfile)
└─ users.routes.js                   ← File 24 (PATCH – add /me/profile route)
```

File 19 · backend/src/modules/lookups/lookups.repo.js

```
javascript
// backend/src/modules/lookups/lookups.repo.js
'use strict';
const pool = require('../../config/db');

async function listDivisions() {
  const [rows] = await pool.query(
    'SELECT id, code, name FROM divisions ORDER BY name ASC',
  );
  return rows;
}

async function searchEquipment(q, limit) {
  const term = '%' + (q || '').trim() + '%';
  const [rows] = await pool.query(
    `SELECT id, equipment_id_code, name, make, model_no, serial_no, type
    FROM equipment
    WHERE equipment_id_code LIKE ?
    OR name LIKE ?
    OR serial_no LIKE ?
    ORDER BY equipment_id_code
    LIMIT ?`,
    [term, term, term, limit],
  );
  return rows;
}

module.exports = { listDivisions, searchEquipment };
```

File 20 · backend/src/modules/lookups/lookups.controller.js

```

javascript
// backend/src/modules/lookups/lookups.controller.js
'use strict';
const repo = require('./lookups.repo');

async function divisions(_req, res, next) {
  try {
    const data = await repo.listDivisions();
    res.json({ data });
  } catch (e) { next(e); }
}

async function equipmentSearch(req, res, next) {
  try {
    const q      = String(req.query.q || '').slice(0, 120);
    const limit = Math.min(Number(req.query.limit) || 20, 50);
    const data  = await repo.searchEquipment(q, limit);
    res.json({ data });
  } catch (e) { next(e); }
}

module.exports = { divisions, equipmentSearch };

```

File 21 · backend/src/modules/lookups/lookups.routes.js

```

javascript
// backend/src/modules/lookups/lookups.routes.js
'use strict';
const router = require('express').Router();
const ctrl = require('./lookups.controller');
const authenticate = require('../../middleware/authenticate');
const { authorize } = require('../../middleware/authorize');

router.get('/divisions',
  authenticate,
  authorize('job_request:create'),
  ctrl.divisions,
);

router.get('/equipment/search',
  authenticate,
  authorize('job_request:create'),
  ctrl.equipmentSearch,
);

module.exports = router;

```

File 22 · backend/src/middleware/rowLevelScope.js (REPLACE)

The Phase 4 version was a no-op stub. Replace it with this real implementation. It branches on the request path — job-requests gets BR-VIS-01 (normal sees own only), job-cards gets BR-VIS-02 (engineer sees own + queue).

```
javascript
// backend/src/middleware/rowLevelScope.js  (REPLACE existing stub)
'use strict';

/**
 * Sets req.scope based on the user's role + the resource being touched.
 * BR-VIS-01 / -02 / -03 / -04 (FINAL-DESC §9.6).
 *
 * For Job Requests:
 *   Normal User    → scope.onlyOwnerId = me.id      (own rows only)
 *   Everyone else → no row filter (read-all rules apply)
 *
 * For Job Cards:
 *   Lab Engineer   → scope.engineerId  = me.id      (own + queue)
 *   Everyone else → no row filter
 */
function rowLevelScope(req, _res, next) {
  const path = req.baseUrl + req.path;
  const me = req.user;
  if (!me) return next();
  req.scope = {};

  if (path.startsWith('/api/v1/job-requests')) {
    const hasReadAll = me.permissions.includes('job_request:read-all');
    if (!hasReadAll) req.scope.onlyOwnerId = me.userId;
  }

  if (path.startsWith('/api/v1/job-cards')) {
    if (me.role === 'LAB_ENGINEER') req.scope.engineerId = me.userId;
  }

  next();
}

module.exports = rowLevelScope;
```

File 23 · backend/src/modules/users/users.controller.js (PATCH)

Existing getMe stays. Add getProfile alongside it. The profile endpoint joins divisions so the form has both the id and the human-readable code+name without a second round-trip.

```
javascript
// backend/src/modules/users/users.controller.js  (ADD getProfile)
'use strict';
const pool = require('../../config/db');

function getMe(req, res) {
  res.json({ data: {
    id: req.user.userId,
    employee_id: req.user.employeeId,
    role: req.user.role,
    permissions: req.user.permissions,
  }});
}
```

```

}

async function getProfile(req, res, next) {
  try {
    const [rows] = await pool.query(
      `SELECT u.id, u.employee_id, u.full_name, u.designation, u.email,
        u.lab_phone, u.room_phone, u.division_id,
        d.code AS division_code, d.name AS division_name
        FROM users u
        LEFT JOIN divisions d ON d.id = u.division_id
        WHERE u.id = ?`,
      [req.user.userId],
    );
    const me = rows[0];
    if (!me) return res.status(404).json({ errors: { user: 'not_found' } });
    res.json({ data: me });
  } catch (e) { next(e); }
}

module.exports = { getMe, getProfile };

```

File 24 · backend/src/modules/users/users.routes.js (PATCH)

```

javascript
// backend/src/modules/users/users.routes.js  (ADD /me/profile)
'use strict';
const router = require('express').Router();
const ctrl = require('./users.controller');
const authenticate = require('../../middleware/authenticate');

router.get('/me',          authenticate, ctrl.getMe);
router.get('/me/profile', authenticate, ctrl.getProfile);

module.exports = router;

```

✓ Verify Step 5

New lookups module is three files.

rowLevelScope.js no longer just calls next() — it actually sets req.scope.

/me/profile route exists alongside /me.

Step 6 — Wire the New Routes in server.js

→ What you do in this step

Six lines of code. Two imports, three `app.use` calls, and you have unblocked every Step 7 onwards test.

File 25 · backend/src/server.js (PATCH)

```
javascript
// backend/src/server.js (PATCH — add the three highlighted lines)
'use strict';
// ... existing imports unchanged ...
const authRoutes = require('./modules/auth/auth.routes');
const usersRoutes = require('./modules/users/users.routes');
// + ADD:
const jobRequestsRoutes = require('./modules/jobRequests/jobRequests.routes');
const jobCardsRoutes = require('./modules/jobCards/jobCards.routes');
const lookupsRoutes = require('./modules/lookups/lookups.routes');

// ... after existing middleware pipeline ...
app.use('/api/v1/auth', authRoutes);
app.use('/api/v1/users', usersRoutes);
// + ADD:
app.use('/api/v1/job-requests', jobRequestsRoutes);
app.use('/api/v1/job-cards', jobCardsRoutes);
app.use('/api/v1/lookups', lookupsRoutes);

// ... 404 + errorHandler unchanged ...
```

✓ Verify Step 6 — first real smoke test

Restart the backend: `npm run dev` inside backend/

`curl -i http://localhost:3000/api/v1/job-requests`

Expected: HTTP/1.1 401 Unauthorized — that is correct! It means the route is mounted and the authenticate middleware fired.

A 404 here means the `app.use` line is missing or the path is wrong.

A 500 here means a JS error in one of the new files — read the pino log line.

Part III — The Code (Frontend)

The frontend follows the same bottom-up discipline as the backend: foundation (permissions, routes, palette, hooks, schemas) before pages. By the end of Step 7 you will have a typed contract between client and server but no visible UI changes yet.

Step 7 — Frontend Foundation

→ What you do in this step

Patch three existing files and create five new ones. After this, nav items will appear in the sidebar for users with the right permissions, and the React Query layer will be ready to talk to the new endpoints.

Folder layout after this step

```
frontend/src/
├── lib/
│   └── permissions.js          ← File 26 (PATCH)
├── App.jsx                    ← File 27 (PATCH)
├── api/
│   ├── jobRequests.api.js     ← File 28 (NEW)
│   ├── jobCards.api.js       ← File 29 (NEW)
│   └── lookups.api.js        ← File 30 (NEW)
├── schemas/
│   ├── jobRequestSchemas.js  ← File 31 (NEW)
│   └── jobCardSchemas.js     ← File 32 (NEW)
└── frontend/
    └── tailwind.config.js     ← File 33 (PATCH)
```

File 26 · frontend/src/lib/permissions.js (PATCH)

The nav-item registry is the single source of truth for what appears in the sidebar. Add two entries — Job Requests gates on read-own (the loosest read permission everyone has), Job Cards gates on read-list (which Normal Users lack, so they will not see it).

```
javascript
// frontend/src/lib/permissions.js (PATCH — add Job Requests + Job Cards)
import {
  LayoutDashboard, Wrench, FileText, ClipboardCheck,
  Search, ScrollText, Users,
} from 'lucide-react';

export const ALL_NAV_ITEMS = [
  { label: 'Dashboard', to: '/dashboard', icon: LayoutDashboard, requires:
'dashboard:view' },
  { label: 'Equipment', to: '/equipment', icon: Wrench, requires:
'equipment:read-list' },
  // + ADD:
  { label: 'Job Requests', to: '/job-requests', icon: FileText, requires:
'job_request:read-own' },
  { label: 'Job Cards', to: '/job-cards', icon: ClipboardCheck, requires: 'job_card:read-
list' },
  // existing items below unchanged
  { label: 'Inquiry', to: '/inquiry', icon: Search, requires:
'inquiry:search-instruments' },
  { label: 'Audit Log', to: '/audit', icon: ScrollText, requires:
'audit_log:read' },
  { label: 'Manage Users', to: '/admin/users', icon: Users, requires: 'user:read-
list' },
];

export function visibleNavItems(permissions) {
  if (!Array.isArray(permissions) || permissions.length === 0) return [];
}
```

```
const owned = new Set(permissions);
return ALL_NAV_ITEMS.filter(item => owned.has(item.requires));
}
```

File 27 · frontend/src/App.jsx (PATCH)

Three new routes wrapped in ProtectedRoute. ProtectedRoute already exists from Phase 4 — it checks the requiredPermission against the auth context and redirects to /403 if missing.

```
javascript
// frontend/src/App.jsx (PATCH – add the three new routes)
import { Route, Routes } from 'react-router-dom';
// ... existing imports unchanged ...
import ProtectedRoute from '@components/ProtectedRoute';
import Layout from '@components/Layout';
// + ADD:
import JobRequestList from '@pages/jobRequests/JobRequestList';
import JobRequestNew from '@pages/jobRequests/JobRequestNew';
import JobCardList from '@pages/jobCards/JobCardList';

// ... inside <Routes>:
<Route path="/job-requests" element={
  <ProtectedRoute requiredPermission="job_request:read-own">
    <Layout><JobRequestList /></Layout>
  </ProtectedRoute>
} />
<Route path="/job-requests/new" element={
  <ProtectedRoute requiredPermission="job_request:create">
    <Layout><JobRequestNew /></Layout>
  </ProtectedRoute>
} />
<Route path="/job-cards" element={
  <ProtectedRoute requiredPermission="job_card:read-list">
    <Layout><JobCardList /></Layout>
  </ProtectedRoute>
} />
```

File 28 · frontend/src/api/jobRequests.api.js

React Query hooks. useJobRequestsList keeps previous data so the table never flashes empty between filter changes. useCreateDraft and useSubmitRequest both invalidate the list so the new row appears immediately after submit.

```
javascript
// frontend/src/api/jobRequests.api.js
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { api } from '@lib/api-client';

const KEY = 'job-requests';

export function useJobRequestsList(filters) {
  return useQuery({
    queryKey: [KEY, filters],
    queryFn: async () => {
      const { data } = await api.get('/job-requests', { params: filters });
    }
  });
}
```

```

    return data;
  },
  keepPreviousData: true,
  staleTime: 30_000,
  refetchOnWindowFocus: false,
});
}

export function useCreateDraft() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (body) => {
      const { data } = await api.post('/job-requests', body);
      return data.data; // { id, request_code, status }
    },
    onSuccess: () => qc.invalidateQueries({ queryKey: [KEY] }),
  });
}

export function useSubmitRequest() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, tnc_version = 'v1' }) => {
      const { data } = await api.post(`/job-requests/${id}/submit`, {
        tnc_accepted: true, tnc_version,
      });
      return data.data; // { id, request_code, status }
    },
    onSuccess: () => qc.invalidateQueries({ queryKey: [KEY] }),
  });
}

```

File 29 · frontend/src/api/jobCards.api.js

```

javascript
// frontend/src/api/jobCards.api.js
import { useQuery } from '@tanstack/react-query';
import { api } from '@lib/api-client';

export function useJobCardsList(filters) {
  return useQuery({
    queryKey: ['job-cards', filters],
    queryFn: async () => {
      const { data } = await api.get('/job-cards', { params: filters });
      return data;
    },
    keepPreviousData: true,
    staleTime: 30_000,
    refetchOnWindowFocus: false,
  });
}

```

File 30 · frontend/src/api/lookups.api.js

Three small hooks: divisions (cached 5 minutes, rarely changes), equipment search (cached 30s, only fires after 2 chars typed), and my profile (cached 1 hour, only refetches on logout/login).

```

javascript
// frontend/src/api/lookups.api.js
import { useQuery } from '@tanstack/react-query';
import { api } from '@lib/api-client';

export function useDivisionsLookup() {
  return useQuery({
    queryKey: ['lookups', 'divisions'],
    queryFn: async () => {
      const { data } = await api.get('/lookups/divisions');
      return data.data; // [{ id, code, name }]
    },
    staleTime: 5 * 60_000, // 5 min – rarely changes
  });
}

export function useEquipmentSearch(q, enabled = true) {
  return useQuery({
    queryKey: ['lookups', 'equipment', q],
    queryFn: async () => {
      const { data } = await api.get('/lookups/equipment/search', {
        params: { q, limit: 20 }
      });
      return data.data;
    },
    enabled: enabled && (q?.length ?? 0) >= 2,
    staleTime: 30_000,
  });
}

export function useMyProfile() {
  return useQuery({
    queryKey: ['me', 'profile'],
    queryFn: async () => {
      const { data } = await api.get('/users/me/profile');
      return data.data;
    }
  });
}

```

```

    },
    staleTime: 60 * 60_000,      // 1 hour
  });
}

```

File 31 · frontend/src/schemas/jobRequestSchemas.js

Mirror of the backend zod schema. Same field names, same constraints. The crucial point: `submitter_*` fields are absent here too — the form literally cannot send them, so BR-JR-06 is enforced by the type system before it is enforced by the `.strict()` backend schema.

```

javascript
// frontend/src/schemas/jobRequestSchemas.js
import { z } from 'zod';

export const accessory = z.object({
  type: z.string().min(1, 'required').max(60),
  name: z.string().min(1, 'required').max(120),
  serial_no: z.string().max(120).optional().or(z.literal('')),
});

// Mirror of backend createSchema – submitter_* fields excluded by design.
export const newRequestSchema = z.object({
  job_category: z.enum(['TME', 'FPE'], { required_error: 'select a category' }),
  job_type: z.enum(['CALIBRATION', 'REPAIR', 'REGISTRATION'], { required_error: 'select a type' }),
  equipment_id: z.number().int().positive().nullable().optional(),
  equipment_name: z.string().min(2, 'min 2 chars').max(200),
  make: z.string().max(120).optional().or(z.literal('')),
  model_no: z.string().max(120).optional().or(z.literal('')),
  serial_no: z.string().max(120).optional().or(z.literal('')),
  equipment_type: z.string().max(60).optional().or(z.literal('')),
  options_description: z.string().max(2000).optional().or(z.literal('')),
  accessories: z.array(accessory).max(20).default([]),
  lab_phone: z.string().max(40).optional().or(z.literal('')),
  room_phone: z.string().max(40).optional().or(z.literal('')),
  division_id: z.number({ invalid_type_error: 'select a division' }).int().positive(),
  subsystem: z.string().max(120).optional().or(z.literal('')),
  project_name: z.string().max(160).optional().or(z.literal('')),
  complaint_description: z.string().min(10, 'describe the issue (min 10 chars)').max(4000),
  remarks: z.string().max(2000).optional().or(z.literal('')),
  equipment_sent_after_repair: z.boolean().default(false),
  priority: z.enum(['LOW', 'MEDIUM', 'HIGH']).default('MEDIUM'),
});

```

File 32 · frontend/src/schemas/jobCardSchemas.js

```

javascript
// frontend/src/schemas/jobCardSchemas.js
import { z } from 'zod';

export const jobCardStatus = z.enum([
  'ASSIGNED', 'IN_PROGRESS', 'COMPLETED', 'VERIFIED_CLOSED', 'REOPENED',
]);

```

File 33 · frontend/tailwind.config.js (PATCH)

Adds status-token colors so we never hard-code hex values inside components. Pair these tokens with `bg-*`, `text-*` utilities in the `StatusPill` component.

```
javascript
// frontend/tailwind.config.js (PATCH - extend theme.extend.colors)
// Add these alongside the existing 11 tokens. Light-only palette.
colors: {
  // ... existing base, ink, border, accent, success, warning, danger ...
  status: {
    draft: { bg: '#F1F5F9', fg: '#334155' }, // slate
    pending: { bg: '#FEF3C7', fg: '#92400E' }, // amber
    approved: { bg: '#EDE9FE', fg: '#5B21B6' }, // violet
    in_progress: { bg: '#DBEAFE', fg: '#1E40AF' }, // blue
    completed: { bg: '#DCFCE7', fg: '#166534' }, // green
    verified_closed: { bg: '#D1FAE5', fg: '#065F46' }, // emerald
    rejected: { bg: '#FEE2E2', fg: '#991B1B' }, // red
    reopened: { bg: '#FFEDD5', fg: '#9A3412' }, // orange
  },
},
```

✓ Verify Step 7

`npm run dev` in `frontend/` starts without errors.

Log in as a Normal User — the sidebar should now show a "Job Requests" item (no "Job Cards" yet, that is correct).

Clicking it lands you on a blank page — the route exists but the `JobRequestList` component is empty until Step 9.

Step 8 — Shared Components

→ What you do in this step

Three small, reusable, side-effect-free components. They are dropped into the job-requests folder but used from both pages. Build these now so the table and list pages in Step 9 compose cleanly.

Folder layout after this step

```
frontend/src/pages/jobRequests/components/
├── StatusPill.jsx      ← File 34
├── PriorityLabel.jsx   ← File 35
└── PaginationBlock.jsx ← File 36
```

File 34 · frontend/src/pages/jobRequests/components/StatusPill.jsx

A pill that maps a backend status enum to a human label and a Tailwind color pair. The map is internal — no caller ever passes class names.

```
jsx
// frontend/src/pages/jobRequests/components/StatusPill.jsx
import React from 'react';

const MAP = {
  DRAFT:      { label: 'Draft',      cls: 'bg-slate-100 text-slate-700' },
  SUBMITTED:  { label: 'Pending',    cls: 'bg-amber-100 text-amber-700' },
  ASSIGNED:   { label: 'Approved',   cls: 'bg-violet-100 text-violet-700' },
  IN_PROGRESS: { label: 'In Progress', cls: 'bg-blue-100 text-blue-700' },
  COMPLETED: { label: 'Completed',   cls: 'bg-green-100 text-green-700' },
  VERIFIED_CLOSED: { label: 'Verified', cls: 'bg-emerald-100 text-emerald-700' },
  REJECTED:   { label: 'Rejected',    cls: 'bg-red-100 text-red-700' },
  REOPENED:   { label: 'Reopened',    cls: 'bg-orange-100 text-orange-700' },
  CANCELLED:  { label: 'Cancelled',   cls: 'bg-slate-100 text-slate-500' },
};

export default function StatusPill({ status }) {
  const m = MAP[status] || { label: status, cls: 'bg-gray-100 text-gray-700' };
  return (
    <span className={`inline-flex items-center px-2.5 py-1 rounded-full text-xs font-medium ${m.cls}`}>
      {m.label}
    </span>
  );
}
```

File 35 · frontend/src/pages/jobRequests/components/PriorityLabel.jsx

```
jsx
// frontend/src/pages/jobRequests/components/PriorityLabel.jsx
import React from 'react';

const MAP = {
  HIGH:      'text-red-600 font-semibold',
```

```
MEDIUM: 'text-amber-600 font-semibold',  
LOW:    'text-green-600 font-semibold',  
};  
  
export default function PriorityLabel({ value }) {  
  const cls = MAP[value] || 'text-gray-700';  
  const label = value  
    ? value.charAt(0) + value.slice(1).toLowerCase()  
    : '-';  
  return <span className={cls}>{label}</span>;  
}
```

File 36 · frontend/src/pages/jobRequests/components/PaginationBlock.jsx

The "1 2 ... 99 100" sliding-window design. The window keeps current ± 1 plus the first two and last two pages, with ellipses for gaps. Page size selector lives inside the same component so any list page gets it for free.

```
jsx
// frontend/src/pages/jobRequests/components/PaginationBlock.jsx
import React from 'react';

/**
 * Pagination block with "1 2 ... 99 100" sliding-window design.
 * Window: 1 neighbour on each side of current + first 2 + last 2 + ellipses.
 */
function buildPages(current, total) {
  if (total <= 7) return Array.from({ length: total }, (_, i) => i + 1);
  const pages = new Set([1, 2, total - 1, total]);
  for (let i = current - 1; i <= current + 1; i += 1) {
    if (i >= 1 && i <= total) pages.add(i);
  }
  const sorted = Array.from(pages).sort((a, b) => a - b);
  const out = [];
  for (let i = 0; i < sorted.length; i += 1) {
    if (i > 0 && sorted[i] - sorted[i - 1] > 1) out.push('...');
    out.push(sorted[i]);
  }
  return out;
}

export default function PaginationBlock({
  page, totalPages, pageSize, onPageChange, onPageSizeChange,
}) {
  const pages = buildPages(page, totalPages);
  return (
    <div className="flex flex-wrap items-center justify-between gap-3 mt-4 px-1">
      <div className="text-sm text-ink-soft">
        Page {page} of {totalPages}
        {onPageSizeChange && (
          <>
            <span className="mx-3 text-gray-300">|</span>
            <label className="mr-2">Rows:</label>
            <select
              value={pageSize}
              onChange={(e) => onPageSizeChange(Number(e.target.value))}
              className="border border-border rounded-md px-2 py-1 text-sm bg-white"
            >
              {[10, 25, 50, 100].map((n) => <option key={n} value={n}>{n}</option>)}
            </select>
          </>
        )}
      </div>

      <div className="flex items-center gap-1">
        <button
          onClick={() => onPageChange(Math.max(1, page - 1))}
          disabled={page <= 1}
          className="px-3 py-1.5 rounded-md border border-border text-sm bg-white
            hover:bg-base-elev disabled:opacity-40 disabled:cursor-not-allowed"
        >Previous</button>

        {pages.map((p, i) => p === '...' ? (
          <span key={'e' + i} className="px-2 text-ink-soft">...</span>
        ) : (
          <button
```

```

        key={p}
        onClick={() => onPageChange(p)}
        className={`min-w-[2.25rem] px-2.5 py-1.5 rounded-md text-sm border ${
          p === page
            ? 'bg-accent text-white border-accent'
            : 'bg-white border-border text-ink hover:bg-base-elev'
        }}
      >{p}</button>
    )}

    <button
      onClick={() => onPageChange(Math.min(totalPages, page + 1))}
      disabled={page >= totalPages}
      className="px-3 py-1.5 rounded-md border border-border text-sm bg-white
        hover:bg-base-elev disabled:opacity-40 disabled:cursor-not-allowed"
      >Next</button>
    </div>
  </div>
);
}

```

✓ Verify Step 8

You cannot see these components alone yet. Just confirm they compile (no red squiggles in your editor) and that ESLint/Vite reports no issues. Visual proof comes after Step 9.

Step 9 — Job Request & Job Card Pages

→ What you do in this step

Eleven files for the three screens. List page is two components plus the page itself. New page is one tncContent helper plus five form sections plus the page. Job Cards list is one table plus the page.

Folder layout after this step (final shape)

```

frontend/src/pages/
├── jobRequests/
│   ├── JobRequestList.jsx           ← File 39
│   ├── JobRequestNew.jsx           ← File 47
│   └── components/
│       ├── StatusFill.jsx          (Step 8 · already done)
│       ├── PriorityLabel.jsx        (Step 8 · already done)
│       ├── PaginationBlock.jsx      (Step 8 · already done)
│       ├── JobRequestFilters.jsx    ← File 37
│       └── JobRequestTable.jsx      ← File 38
│   └── form/
│       ├── tncContent.js           ← File 40
│       ├── Section1JobType.jsx      ← File 41
│       ├── Section2Equipment.jsx    ← File 42
│       ├── Section3Accessories.jsx  ← File 43
│       ├── Section4SubmittedBy.jsx  ← File 44
│       └── Section5TermsAndConditions.jsx ← File 45
├── jobCards/
│   ├── JobCardList.jsx             ← File 46
│   └── components/
│       └── JobCardTable.jsx         ← File 48

```

9A — Job Request list page (3 files)

File 37 · components/JobRequestFilters.jsx

A controlled component. Parent owns the filter state and the URL; this component only renders inputs and emits onChange. Search box uses defaultValue + onChange (uncontrolled) so the React tree never re-renders on every keystroke.

```

jsx
// frontend/src/pages/jobRequests/components/JobRequestFilters.jsx
import React from 'react';
import { Filter, Download } from 'lucide-react';

export default function JobRequestFilters({ value, onChange, total }) {
  const set = (k, v) => onChange({ ...value, [k]: v || undefined, page: 1 });

  return (
    <div className="bg-white border border-border rounded-xl p-4 mb-5">
      <div className="flex flex-wrap items-center gap-3">
        <input
          type="text"
          placeholder="Search by Job ID, Equipment, or Submitted By..."
          defaultValue={value.q || ''}
          onChange={(e) => set('q', e.target.value)}
          className="flex-1 min-w-[260px] border border-border rounded-md px-3 py-2 text-sm
            placeholder:text-ink-soft focus:outline-none focus:ring-2 focus:ring-
accent/40"

```

```

/>

<select
  value={value.type || ''}
  onChange={(e) => set('type', e.target.value)}
  className="border border-border rounded-md px-3 py-2 text-sm bg-white min-w-[140px]"
>
  <option value="">All Types</option>
  <option value="CALIBRATION">Calibration</option>
  <option value="REPAIR">Repair</option>
  <option value="REGISTRATION">Registration</option>
</select>

<select
  value={value.status || ''}
  onChange={(e) => set('status', e.target.value)}
  className="border border-border rounded-md px-3 py-2 text-sm bg-white min-w-[150px]"
>
  <option value="">All Statuses</option>
  <option value="DRAFT">Draft</option>
  <option value="SUBMITTED">Pending</option>
  <option value="ASSIGNED">Approved</option>
  <option value="IN_PROGRESS">In Progress</option>
  <option value="COMPLETED">Completed</option>
  <option value="VERIFIED_CLOSED">Verified</option>
  <option value="REJECTED">Rejected</option>
  <option value="REOPENED">Reopened</option>
</select>
</div>

<div className="flex items-center justify-between mt-3">
  <div className="flex items-center gap-2">
    <button
      type="button"
      className="inline-flex items-center gap-1.5 border border-border rounded-md px-3 py-
1.5
        text-sm bg-white hover:bg-base-elev"
    ><Filter size={14}/> Advanced Filters</button>
    <button
      type="button"
      disabled
      title="Coming soon"
      className="inline-flex items-center gap-1.5 border border-border rounded-md px-3 py-
1.5
        text-sm bg-white text-ink-soft opacity-60 cursor-not-allowed"
    ><Download size={14}/> Export</button>
  </div>
  <div className="text-sm text-ink-soft">
    {typeof total === 'number' ? `Showing ${total} of ${total} requests` : 'Loading...'}
  </div>
</div>
</div>
  );
}

```

File 38 · components/JobRequestTable.jsx

Pure presentation. Receives rows + loading flag, renders a skeleton when loading, an empty state when no rows, and the table otherwise. No data fetching here.

```

jsx
// frontend/src/pages/jobRequests/components/JobRequestTable.jsx
import React from 'react';
import { Link } from 'react-router-dom';
import StatusPill from './StatusPill';
import PriorityLabel from './PriorityLabel';

function fmtDate(s) {
  if (!s) return '-';
  return String(s).slice(0, 10);
}

function prettyType(t) {
  if (!t) return '-';
  return t.charAt(0) + t.slice(1).toLowerCase();
}

export default function JobRequestTable({ rows, loading }) {
  if (loading) {
    return (
      <div className="bg-white border border-border rounded-xl overflow-hidden">
        {Array.from({ length: 6 }).map((_, i) => (
          <div key={i} className="h-14 animate-pulse bg-base-elev/40 border-b border-border" />
        ))}
      </div>
    );
  }
  if (!rows.length) {
    return (
      <div className="bg-white border border-border rounded-xl p-12 text-center">
        <p className="text-ink-soft">No requests yet.</p>
      </div>
    );
  }
  return (
    <div className="bg-white border border-border rounded-xl overflow-hidden">
      <table className="min-w-full divide-y divide-border text-sm">
        <thead className="bg-base-elev/50">
          <tr className="text-left text-xs uppercase tracking-wide text-ink-soft">
            <th className="px-6 py-3">Job ID</th>
            <th className="px-6 py-3">Equipment</th>
            <th className="px-6 py-3">Type</th>
            <th className="px-6 py-3">Division</th>
            <th className="px-6 py-3">Submitted By</th>
            <th className="px-6 py-3">Date</th>
            <th className="px-6 py-3">Priority</th>
            <th className="px-6 py-3">Status</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-border">
          {rows.map((r) => (
            <tr key={r.id} className="hover:bg-base-elev/40">
              <td className="px-6 py-4">
                <Link to={` /job-requests/${r.id}`} className="text-accent hover:underline font-medium">
                  {r.request_code}
                </Link>
              </td>
              <td className="px-6 py-4 text-ink">{r.equipment_name}</td>
              <td className="px-6 py-4 text-ink">{prettyType(r.job_type)}</td>

```

```

        <td className="px-6 py-4 text-ink">{r.division_code || '-'}</td>
        <td className="px-6 py-4 text-ink">{r.submitted_by_name || '-'}</td>
        <td className="px-6 py-4 text-ink-soft">{fmtDate(r.submitted_at ||
r.created_at)}</td>
        <td className="px-6 py-4"><PriorityLabel value={r.priority} /></td>
        <td className="px-6 py-4"><StatusPill status={r.status} /></td>
    </tr>
  )}}
</tbody>
</table>
</div>
);
}

```

File 39 · JobRequestList.jsx

The page that ties everything together. Filters are bidirectional with the URL (useSearchParams) so the browser back button and bookmarking just work. The "+ New Job Request" CTA shows only if the user has `job_request:create`.

```

jsx
// frontend/src/pages/jobRequests/JobRequestList.jsx
import React, { useMemo } from 'react';
import { Link, useSearchParams } from 'react-router-dom';
import { Plus } from 'lucide-react';
import { useAuth } from '@lib/auth-context';
import { useJobRequestsList } from '@api/jobRequests.api';
import JobRequestFilters from './components/JobRequestFilters';
import JobRequestTable from './components/JobRequestTable';
import PaginationBlock from './components/PaginationBlock';

function paramsToFilters(sp) {
  const f = {
    q: sp.get('q') || undefined,
    type: sp.get('type') || undefined,
    status: sp.get('status') || undefined,
    priority: sp.get('priority') || undefined,
    division_id: sp.get('division_id') || undefined,
    date_from: sp.get('date_from') || undefined,
    date_to: sp.get('date_to') || undefined,
    sort: sp.get('sort') || '-created_at',
    page: Number(sp.get('page')) || 1,
    page_size: Number(sp.get('page_size')) || 25,
  };
  Object.keys(f).forEach((k) => f[k] === undefined && delete f[k]);
  return f;
}

function filtersToParams(f) {
  const o = {};
  Object.entries(f).forEach(([k, v]) => {
    if (v !== undefined && v !== '' && v !== null) o[k] = String(v);
  });
  return o;
}

export default function JobRequestList() {
  const { user } = useAuth();
  const [sp, setSp] = useSearchParams();
  const filters = useMemo(() => paramsToFilters(sp), [sp]);

  const setFilters = (next) => setSp(filtersToParams(next));

  const { data, isLoading } = useJobRequestsList(filters);
  const rows = data?.data || [];
  const pag = data?.pagination || { page: 1, total_pages: 1, total: 0 };

  const canCreate = user?.permissions?.includes('job_request:create');

  return (
    <div className="p-6 max-w-[1400px] mx-auto">
      <div className="flex items-start justify-between mb-5">
        <div>
          <h1 className="text-2xl font-semibold text-ink">Job Requests</h1>
          <p className="text-sm text-ink-soft mt-1">Manage equipment calibration and maintenance requests</p>
        </div>
        {canCreate && (

```

```
        <Link
          to="/job-requests/new"
          className="inline-flex items-center gap-2 bg-accent hover:bg-accent-hover text-white
                    rounded-lg px-4 py-2 text-sm font-medium shadow-sm"
        ><Plus size={16}/> New Job Request</Link>
      )}
    </div>

    <JobRequestFilters value={filters} onChange={setFilters} total={pag.total} />

    <JobRequestTable rows={rows} loading={isLoading} />

    <PaginationBlock
      page={pag.page}
      totalPages={pag.total_pages}
      pageSize={filters.page_size}
      onPageChange={(p) => setFilters({ ...filters, page: p })}
      onPageSizeChange={(n) => setFilters({ ...filters, page_size: n, page: 1 })}
    />
  </div>
);
}
```

9B — New Job Request form (7 files)

File 40 · components/form/tncContent.js

The verbatim six T&C strings from the design. Version constant is TNC_VERSION = "v1" — the server stores this alongside the accepted_at timestamp so a later T&C revision can be detected.

```
javascript
// frontend/src/pages/jobRequests/components/form/tncContent.js
// Verbatim T&C strings from the design.
export const TNC_VERSION = 'v1';

export const TNC_ITEMS = [
  { id: 'tnc1', label: 'I confirm that all equipment details provided are accurate and complete to the best of my knowledge.' },
  { id: 'tnc2', label: 'I understand that the equipment must be delivered to the calibration lab in proper working condition with all necessary accessories.' },
  { id: 'tnc3', label: 'I acknowledge that the calibration timeline begins only after the equipment is received and inspected by the lab.' },
  { id: 'tnc4', label: 'I agree to coordinate with the assigned lab engineer for any additional information or testing requirements.' },
  { id: 'tnc5', label: 'I accept that equipment found to be damaged or beyond repair will be returned with appropriate documentation and recommendations.' },
  { id: 'tnc6', label: 'I understand that urgency requests will be handled based on lab capacity and must be justified with proper authorization.' },
];
```

File 41 · components/form/Section1JobType.jsx

```
jsx
// frontend/src/pages/jobRequests/components/form/Section1JobType.jsx
import React from 'react';

export default function Section1JobType({ form }) {
  const { register, formState: { errors } } = form;
  return (
    <section className="bg-white border border-border rounded-xl p-6">
      <h2 className="text-lg font-semibold mb-4 text-ink">1. Job Type Selection</h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
        <div>
          <label className="text-sm text-ink-soft">Job Category <span className="text-danger">*</span></label>
          <select {...register('job_category')}
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm bg-white">
            <option value="">Select category</option>
            <option value="TME">T&ME</option>
            <option value="FPE">F&PE</option>
          </select>
          {errors.job_category && <p className="text-xs text-danger mt-1">{errors.job_category.message}</p>}
        </div>
        <div>
          <label className="text-sm text-ink-soft">Job Type <span className="text-danger">*</span></label>
          <select {...register('job_type')}
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm bg-white">
            <option value="">Select type</option>
            <option value="CALIBRATION">Calibration</option>
            <option value="REPAIR">Repair</option>
            <option value="REGISTRATION">Registration</option>
          </select>
        </div>
      </div>
    </section>
  );
}
```

```
        {errors.job_type && <p className="text-xs text-danger mt-  
1">{errors.job_type.message}</p>}  
        </div>  
    </div>  
</section>  
);  
}
```

File 42 · components/form/Section2Equipment.jsx

Typeahead-style equipment picker. Querying starts at 2 characters. Picking a match auto-fills the read-aloud fields (make, model, serial, type) so the user does not re-type what the lab already has on file.

```

jsx
// frontend/src/pages/jobRequests/components/form/Section2Equipment.jsx
import React, { useState } from 'react';
import { useEquipmentSearch } from '@api/lookups.api';

export default function Section2Equipment({ form }) {
  const { register, setValue, formState: { errors } } = form;
  const [q, setQ] = useState('');
  const { data: matches = [] } = useEquipmentSearch(q);

  const pick = (eq) => {
    setValue('equipment_id', eq.id, { shouldValidate: true });
    setValue('equipment_name', eq.name);
    setValue('make', eq.make || '');
    setValue('model_no', eq.model_no || '');
    setValue('serial_no', eq.serial_no || '');
    setValue('equipment_type', eq.type || '');
    setQ('');
  };

  return (
    <section className="bg-white border border-border rounded-xl p-6">
      <h2 className="text-lg font-semibold mb-4 text-ink">2. Equipment Details</h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
        <div className="md:col-span-2 relative">
          <label className="text-sm text-ink-soft">Equipment ID <span className="text-
danger">*</span></label>
          <input
            type="text"
            value={q}
            onChange={(e) => setQ(e.target.value)}
            placeholder="Search or enter equipment ID"
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
          {q.length >= 2 && matches.length > 0 && (
            <ul className="absolute z-10 mt-1 w-full bg-white border border-border rounded-md
shadow-sm max-h-60 overflow-auto">
              {matches.map((m) => (
                <li key={m.id}>
                  <button type="button" onClick={() => pick(m)}
                    className="w-full text-left px-3 py-2 text-sm hover:bg-base-elev">
                    <span className="font-medium">{m.equipment_id_code}</span> - {m.name}
                  </button>
                </li>
              )}}
            </ul>
          )}
        </div>

        <Field label="Equipment Name *" name="equipment_name" register={register}
err={errors.equipment_name} />
        <Field label="Make" name="make" register={register}
err={errors.make} />
        <Field label="Model No." name="model_no" register={register}
err={errors.model_no} />
        <Field label="Serial No." name="serial_no" register={register}
err={errors.serial_no} />
        <Field label="Equipment Type" name="equipment_type" register={register}
err={errors.equipment_type} />

        <div className="md:col-span-2">

```

```
        <label className="text-sm text-ink-soft">Options / Description</label>
        <textarea rows={3} {...register('options_description')}
          className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
      </div>
    </div>
  </section>
);
}

function Field({ label, name, register, err }) {
  return (
    <div>
      <label className="text-sm text-ink-soft">{label}</label>
      <input type="text" {...register(name)}
        className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
      {err && <p className="text-xs text-danger mt-1">{err.message}</p>}
    </div>
  );
}
```

File 43 · components/form/Section3Accessories.jsx

useFieldArray from react-hook-form gives us an editable dynamic list with zero state plumbing. Empty list is the legal default — accessories are optional.

```
jsx
// frontend/src/pages/jobRequests/components/form/Section3Accessories.jsx
import React from 'react';
import { useFieldArray } from 'react-hook-form';
import { Plus, X } from 'lucide-react';

export default function Section3Accessories({ form }) {
  const { control, register } = form;
  const { fields, append, remove } = useFieldArray({ control, name: 'accessories' });

  return (
    <section className="bg-white border border-border rounded-xl p-6">
      <h2 className="text-lg font-semibold mb-1 text-ink">3. Accessories</h2>
      <p className="text-sm text-ink-soft mb-4">
        Add any accessories or additional instruments required with the main equipment
      </p>

      <div className="border border-dashed border-border rounded-lg p-4">
        <p className="text-sm font-medium mb-3 text-ink">Add Accessory (Optional)</p>

        {fields.map((f, idx) => (
          <div key={f.id} className="grid grid-cols-1 md:grid-cols-12 gap-3 mb-3 items-end">
            <div className="md:col-span-4">
              <label className="text-xs text-ink-soft">Accessory Type</label>
              <input {...register(`accessories.${idx}.type`)}
                className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
            </div>
            <div className="md:col-span-4">
              <label className="text-xs text-ink-soft">Accessory Name</label>
              <input {...register(`accessories.${idx}.name`)}
                className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
            </div>
            <div className="md:col-span-3">
              <label className="text-xs text-ink-soft">Serial No.</label>
              <input {...register(`accessories.${idx}.serial_no`)}
                className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
            </div>
            <div className="md:col-span-1">
              <button type="button" onClick={() => remove(idx)}
                className="text-ink-soft hover:text-danger p-2">X size={16}</button>
            </div>
          </div>
        ))}

        <button type="button"
          onClick={() => append({ type: '', name: '', serial_no: '' })}
          className="inline-flex items-center gap-1.5 bg-base-elev hover:bg-base text-ink-soft
            rounded-md px-3 py-1.5 text-sm border border-border">
          <Plus size={14}> Add Accessory
        </button>
      </div>
    </section>
  );
}
```

File 44 · components/form/Section4SubmittedBy.jsx

The four critical identity fields (name, employee id, designation, email) come from useMyProfile and are rendered as disabled inputs. The form schema does not include them so even if a malicious user enables them in DevTools, they are stripped at the validator boundary.

```
jsx
// frontend/src/pages/jobRequests/components/form/Section4SubmittedBy.jsx
import React, { useEffect } from 'react';
import { useMyProfile, useDivisionsLookup } from '@api/lookups.api';

export default function Section4SubmittedBy({ form }) {
  const { register, setValue, formState: { errors } } = form;
  const { data: me } = useMyProfile();
  const { data: divisions } = useDivisionsLookup();

  // Pre-fill division + phones from profile (editable)
  useEffect(() => {
    if (me) {
      if (me.division_id) setValue('division_id', me.division_id);
      if (me.lab_phone) setValue('lab_phone', me.lab_phone);
      if (me.room_phone) setValue('room_phone', me.room_phone);
    }
  }, [me, setValue]);

  if (!me) {
    return (
      <section className="bg-white border border-border rounded-xl p-6">
        <h2 className="text-lg font-semibold mb-4 text-ink">4. Submitted By</h2>
        <p className="text-sm text-ink-soft">Loading profile...</p>
      </section>
    );
  }

  return (
    <section className="bg-white border border-border rounded-xl p-6">
      <h2 className="text-lg font-semibold mb-4 text-ink">4. Submitted By</h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
        <ReadOnly label="Name *" value={me.full_name}/>
        <ReadOnly label="SAC Employee ID *" value={me.employee_id}/>
        <ReadOnly label="Designation *" value={me.designation}/>
        <ReadOnly label="Email *" value={me.email}/>

        <div>
          <label className="text-sm text-ink-soft">Lab Phone</label>
          <input type="text" {...register('lab_phone')}
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
        </div>
        <div>
          <label className="text-sm text-ink-soft">Room Phone</label>
          <input type="text" {...register('room_phone')}
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
        </div>

        <div>
          <label className="text-sm text-ink-soft">Division <span className="text-
danger">*</span></label>
          <select
            {...register('division_id', { valueAsNumber: true })}
            className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm bg-white">
            <option value="">Select division</option>
            {(divisions || []).map(d =>
              <option key={d.id} value={d.id}>{d.code} – {d.name}</option>
            )}
          </select>
        </div>
      </div>
    </section>
  );
}
```

```

        {errors.division_id && <p className="text-xs text-danger mt-
1">{errors.division_id.message}</p>}
      </div>
      <div>
        <label className="text-sm text-ink-soft">Subsystem</label>
        <input type="text" {...register('subsystem')}
          className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
      </div>

      <div className="md:col-span-2">
        <label className="text-sm text-ink-soft">Project</label>
        <input type="text" {...register('project_name')}
          className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm"
          placeholder="Enter project name" />
      </div>

      <div className="md:col-span-2">
        <label className="text-sm text-ink-soft">Complaint Description <span className="text-
danger">*</span></label>
        <textarea rows={4} {...register('complaint_description')}
          placeholder="Describe the issue or requirements in detail"
          className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
        {errors.complaint_description && <p className="text-xs text-danger mt-
1">{errors.complaint_description.message}</p>}
      </div>

      <div className="md:col-span-2">
        <label className="text-sm text-ink-soft">Remarks</label>
        <textarea rows={2} {...register('remarks')}
          placeholder="Additional remarks or notes"
          className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm" />
      </div>

      <div className="md:col-span-2 flex items-center gap-2">
        <input type="checkbox" {...register('equipment_sent_after_repair')}
          id="esar" className="rounded border-border" />
        <label htmlFor="esar" className="text-sm text-ink">Equipment sent after repair</label>
      </div>
    </div>
  </section>
);
}

function ReadOnly({ label, value }) {
  return (
    <div>
      <label className="text-sm text-ink-soft">{label}</label>
      <input type="text" value={value || ''} disabled
        className="mt-1 w-full border border-border rounded-md px-3 py-2 text-sm bg-base-elev
text-ink" />
    </div>
  );
}

```

File 45 · components/form/Section5TermsAndConditions.jsx

Six checkboxes. The submit button is disabled until every single one is checked — this is the FE half of the double-lock. The BE half is the zod literal(true) check on tnc_accepted.

```

jsx
// frontend/src/pages/jobRequests/components/form/Section5TermsAndConditions.jsx
import React from 'react';
import { AlertCircle } from 'lucide-react';
import { TNC_ITEMS } from './tncContent';

export default function Section5TermsAndConditions({ checks, setCheck }) {
  const accepted = TNC_ITEMS.filter(t => checks[t.id]).length;
  return (
    <section className="bg-white border border-border rounded-xl p-6">
      <div className="flex items-center gap-2 mb-1">
        <AlertCircle size={18} className="text-accent" />
        <h2 className="text-lg font-semibold text-ink">5. Terms and Conditions</h2>
      </div>
      <p className="text-sm text-ink-soft mb-4">
        Please read and accept all terms and conditions before submitting your request
      </p>

      <ul className="space-y-3">
        {TNC_ITEMS.map((t, i) => (
          <li key={t.id}
            className="border border-border rounded-md px-4 py-3 flex items-start gap-3">
              <input type="checkbox"
                id={t.id}
                checked={!!checks[t.id]}
                onChange={(e) => setCheck(t.id, e.target.checked)}
                className="mt-1 h-4 w-4 rounded border-border" />
              <label htmlFor={t.id} className="text-sm text-ink">
                <span className="font-semibold">T&C {i + 1}</span> {t.label}
              </label>
            </li>
          </li>
        ))}
      </ul>

      <div className="mt-4 bg-amber-50 border border-amber-200 rounded-md px-4 py-3 text-sm text-amber-800">
        <AlertCircle size={14} className="inline mr-1" />
        You must accept all terms and conditions to submit this request.
        <span className="font-semibold ml-1">{accepted}/{TNC_ITEMS.length} accepted</span>
      </div>
    </section>
  );
}

```

File 46 · jobRequests/JobRequestNew.jsx

Orchestrates the five sections. handleSubmit calls createDraft, then submit. If the second call fails, the draft remains — the user can retry without re-typing everything. Toast feedback comes from sonner (already in the Phase 5 stack).

```
jsx
// frontend/src/pages/jobRequests/JobRequestNew.jsx
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { ArrowLeft, Save, Send } from 'lucide-react';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { toast } from 'sonner';

import { newRequestSchema } from '@schemas/jobRequestSchemas';
import { useCreateDraft, useSubmitRequest } from '@api/jobRequests.api';
import { TNC_ITEMS, TNC_VERSION } from './components/form/tncContent';

import Section1JobType from './components/form/Section1JobType';
import Section2Equipment from './components/form/Section2Equipment';
import Section3Accessories from './components/form/Section3Accessories';
import Section4SubmittedBy from './components/form/Section4SubmittedBy';
import Section5TermsAndConditions from './components/form/Section5TermsAndConditions';

export default function JobRequestNew() {
  const navigate = useNavigate();
  const createDraft = useCreateDraft();
  const submitReq = useSubmitRequest();

  const form = useForm({
    resolver: zodResolver(newRequestSchema),
    defaultValues: {
      accessories: [],
      equipment_sent_after_repair: false,
      priority: 'MEDIUM',
    },
  });

  const [tnc, setTnc] = useState({});
  const setCheck = (id, v) => setTnc((p) => ({ ...p, [id]: v }));
  const allAccepted = TNC_ITEMS.every((t) => tnc[t.id]);

  const onSaveDraft = form.handleSubmit(async (values) => {
    try {
      const r = await createDraft.mutateAsync(values);
      toast.success(`Saved as draft (${r.request_code})`);
      navigate('/job-requests');
    } catch (e) {
      toast.error(e?.response?.data?.errors?._global || 'Failed to save draft');
    }
  });

  const onSubmit = form.handleSubmit(async (values) => {
    if (!allAccepted) {
      toast.warning('Please accept all six terms and conditions');
      return;
    }
    try {
      const created = await createDraft.mutateAsync(values);
      await submitReq.mutateAsync({ id: created.id, tnc_version: TNC_VERSION });
      toast.success(`Submitted (${created.request_code})`);
      navigate('/job-requests');
    } catch (e) {

```

```

    toast.error(e?.response?.data?.errors?._global || 'Failed to submit');
  }
});

return (
  <div className="p-6 max-w-[1400px] mx-auto">
    <Link to="/job-requests" className="inline-flex items-center gap-2 text-sm text-ink-soft
hover:text-ink mb-3">
      <ArrowLeft size={14}/> Back to Job Requests
    </Link>
    <h1 className="text-2xl font-semibold text-ink">New Job Request</h1>
    <p className="text-sm text-ink-soft mt-1 mb-6">
      Submit a new calibration, repair, or registration request
    </p>

    <form className="space-y-6">
      <Section1JobType form={form} />
      <Section2Equipment form={form} />
      <Section3Accessories form={form} />
      <Section4SubmittedBy form={form} />
      <Section5TermsAndConditions checks={tnc} setCheck={setCheck} />

      <div className="flex items-center justify-end gap-3 pt-2">
        <Link to="/job-requests"
          className="px-4 py-2 rounded-lg border border-border bg-white hover:bg-base-elev
text-sm">
          Cancel
        </Link>
        <button type="button" onClick={onSaveDraft}
          disabled={createDraft.isPending}
          className="inline-flex items-center gap-2 px-4 py-2 rounded-lg border border-border
bg-white
          hover:bg-base-elev text-sm">
          <Save size={14}/> Save as Draft
        </button>
        <button type="button" onClick={onSubmit}
          disabled={!allAccepted || submitReq.isPending || createDraft.isPending}
          className="inline-flex items-center gap-2 px-4 py-2 rounded-lg text-sm text-white
bg-accent hover:bg-accent-hover disabled:opacity-40 disabled:cursor-not-
allowed">
          <Send size={14}/> Submit Request
        </button>
      </div>
    </form>
  </div>
);
}

```

9C — Job Card list page (2 files)

File 47 · jobCards/components/JobCardTable.jsx

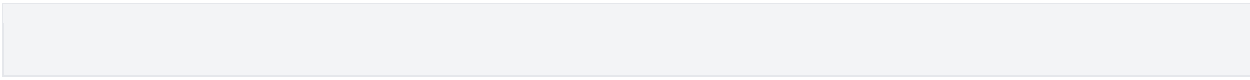
```

jsx
// frontend/src/pages/jobCards/components/JobCardTable.jsx
import React from 'react';
import { Link } from 'react-router-dom';
import StatusPill from '../../jobRequests/components/StatusPill';

function fmtDate(s) { return s ? String(s).slice(0, 10) : '-'; }

export default function JobCardTable({ rows, loading }) {
  if (loading) {
    return (
      <div className="bg-white border border-border rounded-xl overflow-hidden">
        {Array.from({ length: 6 }).map((_, i) => (
          <div key={i} className="h-14 animate-pulse bg-base-elev/40 border-b border-border" />
        ))}
      </div>
    );
  }
  if (!rows.length) {
    return (
      <div className="bg-white border border-border rounded-xl p-12 text-center">
        <p className="text-ink-soft">No job cards yet.</p>
      </div>
    );
  }
  return (
    <div className="bg-white border border-border rounded-xl overflow-hidden">
      <table className="min-w-full divide-y divide-border text-sm">
        <thead className="bg-base-elev/50">
          <tr className="text-left text-xs uppercase tracking-wide text-ink-soft">
            <th className="px-6 py-3">Job Card ID</th>
            <th className="px-6 py-3">Job Request ID</th>
            <th className="px-6 py-3">Equipment</th>
            <th className="px-6 py-3">Assigned Engineer</th>
            <th className="px-6 py-3">Status</th>
            <th className="px-6 py-3">Start Date</th>
            <th className="px-6 py-3">Due Date</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-border">
          {rows.map((r) => (
            <tr key={r.id} className="hover:bg-base-elev/40">
              <td className="px-6 py-4">
                <Link to={` /job-cards/${r.id}`} className="text-accent hover:underline font-medium">
                  {r.card_code}
                </Link>
              </td>
              <td className="px-6 py-4 text-ink-soft">{r.request_code} ||
                `#${r.job_request_id}`</td>
              <td className="px-6 py-4 text-ink">{r.equipment_name}</td>
              <td className="px-6 py-4 text-ink">{r.assigned_engineer_name} || '- '</td>
              <td className="px-6 py-4"><StatusPill status={r.status} /></td>
              <td className="px-6 py-4 text-ink-soft">{fmtDate(r.start_date)}</td>
              <td className="px-6 py-4 text-ink-soft">{fmtDate(r.due_date)}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```



File 48 · jobCards/JobCardList.jsx

```

jsx
// frontend/src/pages/jobCards/JobCardList.jsx
import React, { useMemo } from 'react';
import { useSearchParams } from 'react-router-dom';
import { Filter, Download } from 'lucide-react';
import { useJobCardsList } from '@api/jobCards.api';
import JobCardTable from '../components/JobCardTable';
import PaginationBlock from '../jobRequests/components/PaginationBlock';

function paramsToFilters(sp) {
  const f = {
    q: sp.get('q') || undefined,
    status: sp.get('status') || undefined,
    sort: sp.get('sort') || '-created_at',
    page: Number(sp.get('page')) || 1,
    page_size: Number(sp.get('page_size')) || 25,
  };
  Object.keys(f).forEach((k) => f[k] === undefined && delete f[k]);
  return f;
}

function filtersToParams(f) {
  const o = {};
  Object.entries(f).forEach(([k, v]) => {
    if (v !== undefined && v !== '' && v !== null) o[k] = String(v);
  });
  return o;
}

export default function JobCardList() {
  const [sp, setSp] = useSearchParams();
  const filters = useMemo(() => paramsToFilters(sp), [sp]);
  const setFilters = (next) => setSp(filtersToParams(next));

  const { data, isLoading } = useJobCardsList(filters);
  const rows = data?.data || [];
  const pag = data?.pagination || { page: 1, total_pages: 1, total: 0 };

  return (
    <div className="p-6 max-w-[1400px] mx-auto">
      <h1 className="text-2xl font-semibold text-ink">Job Cards</h1>
      <p className="text-sm text-ink-soft mt-1 mb-5">Track and manage job execution and progress</p>

      <div className="bg-white border border-border rounded-xl p-4 mb-5">
        <div className="flex flex-wrap items-center gap-3">
          <input type="text"
            placeholder="Search by Job Card ID, Equipment, or Engineer..."
            defaultValue={filters.q || ''}
            onChange={(e) => setFilters({ ...filters, q: e.target.value || undefined, page: 1
          }}}
          className="flex-1 min-w-[260px] border border-border rounded-md px-3 py-2 text-sm"
        />
        <select value={filters.status || ''}
          onChange={(e) => setFilters({ ...filters, status: e.target.value || undefined, page:
1 }}}
          className="border border-border rounded-md px-3 py-2 text-sm bg-white min-w-
[160px]">
          <option value="">All Statuses</option>
          <option value="ASSIGNED">Assigned</option>
          <option value="IN_PROGRESS">In Progress</option>
          <option value="COMPLETED">Completed</option>
          <option value="VERIFIED_CLOSED">Verified</option>
          <option value="REOPENED">Reopened</option>

```

```

        </select>
      </div>
      <div className="flex items-center justify-between mt-3">
        <div className="flex items-center gap-2">
          <button type="button"
            className="inline-flex items-center gap-1.5 border border-border rounded-md px-3
py-1.5 text-sm bg-white hover:bg-base-elev">
            <Filter size={14}/> Advanced Filters</button>
          <button type="button" disabled
            className="inline-flex items-center gap-1.5 border border-border rounded-md px-3
py-1.5 text-sm bg-white text-ink-soft opacity-60 cursor-not-allowed">
            <Download size={14}/> Export</button>
          </div>
          <div className="text-sm text-ink-soft">
            {typeof pag.total === 'number' ? `Showing ${pag.total} of ${pag.total} job cards` :
'Loading...'}
          </div>
        </div>
      </div>

      <JobCardTable rows={rows} loading={isLoading} />

      <PaginationBlock
        page={pag.page} totalPages={pag.total_pages}
        pageSize={filters.page_size}
        onPageChange={p => setFilters({ ...filters, page: p })}
        onPageSizeChange={n => setFilters({ ...filters, page_size: n, page: 1 })}
      />
    </div>
  );
}

```

✓ Verify Step 9

Reload the browser. The two pages now render.

As a Normal User you see "Job Requests" in the nav but not "Job Cards".

As a Lab In-Charge you see both.

Filling the form, accepting all six T&C, and clicking Submit produces a toast with the new JR code and a row in the list.

Part IV — Run & Smoke Test

You have finished typing. This part proves the slice works end-to-end. Pre-flight checks first, then start the backend and frontend, then walk twelve curl commands that cover every acceptance criterion.

Pre-flight Checks

Toolchain

Tool	Expected	How to check
Node.js	≥ 18.0.0	node -v
npm	≥ 9.0.0	npm -v
MySQL CLI	≥ 8.0.0	mysql --version
MySQL server	reachable	mysql -u cmcmis_dev -p -e "SELECT VERSION()"
curl	any	curl --version
jq (optional)	any	jq --version

Environment variables (backend/.env)

```
env
# backend/.env (these must already exist from Phase 4)
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_NAME=cmcmis_dev_db
DB_USER=cmcmis_dev
DB_PASSWORD=*****
JWT_ACCESS_SECRET=... # 64+ chars
JWT_REFRESH_SECRET=... # 64+ chars
COOKIE_DOMAIN=localhost
CORS_ORIGIN=http://localhost:5173
LOG_LEVEL=debug
```

Environment variables (frontend/.env)

```
env
# frontend/.env
VITE_API_BASE_URL=http://localhost:3000/api/v1
```

! If any of these are missing

Stop. Do not start the servers — you will chase phantom errors for an hour.
Copy backend/.env.example → backend/.env, fill in the secrets.

For local dev, JWT secrets can be any random string ≥ 64 chars: openssl rand -hex 32 produces one.

Starting the Servers

Step 1 — Apply migrations

You have already done this during Step 1 of the build map, but re-run them to confirm idempotency.

```
bash
# from project root
cd backend
mysql -u cmcmis_dev -p cmcmis_dev_db < db/migrations/0010_phase6_columns.sql
mysql -u cmcmis_dev -p cmcmis_dev_db < db/migrations/0011_phase6_indexes.sql
mysql -u cmcmis_dev -p cmcmis_dev_db < db/migrations/0012_phase6_status_history.sql
# Expected: zero output, zero errors
```

Step 2 — Backend dev server

```
bash
# in a terminal tab
cd backend
npm install          # if you have not since Phase 4
npm run dev
# Expected log lines:
# [pino] INFO  server listening on http://localhost:3000
# [pino] INFO  db pool ready (10 connections)
# [pino] INFO  routes mounted: /api/v1/auth, /api/v1/users, /api/v1/equipment,
#                                     /api/v1/job-requests, /api/v1/job-cards, /api/v1/lookups
```

Step 3 — Frontend dev server

```
bash
# in a SECOND terminal tab
cd frontend
npm install          # if you have not since Phase 5
npm run dev
# Expected output:
# VITE v5.x.x  ready in 800 ms
# → Local:   http://localhost:5173/
# → Network: use --host to expose
```

Step 4 — Authenticate once, save the cookies

Phase 4 uses httpOnly cookies for JWT. We use curl with -c (write cookie jar) for the login call and -b (read cookie jar) for everything else.

```
bash
# Login as a Normal User (replace with a real test account)
curl -i -c /tmp/normal.cookies \
  -H "Content-Type: application/json" \
  -X POST http://localhost:3000/api/v1/auth/login \
  -d '{"employeeId":"EMP1001","password":"Passw0rd!"}'
# Expected: HTTP/1.1 200 OK
# Expected: Set-Cookie: cmcmis_access=...; HttpOnly; SameSite=Strict
# Expected body: {"data":{"id":...,"employee_id":"EMP1001","role":"NORMAL_USER",...}}
```

```
# Repeat for a Lab In-Charge user
curl -i -c /tmp/lic.cookies \
  -H "Content-Type: application/json" \
  -X POST http://localhost:3000/api/v1/auth/login \
  -d '{"employeeId":"EMP2001","password":"Passw0rd!"}'
```

Smoke #1 — Role-scoped list (A1: Normal sees only own)

✓ **Pass when**

Smoke #2 — Read-all visibility (A2: LIC sees everything)

Smoke #3 — View-Only blocked from create (A3)

Smoke #4 — Submitter is set server-side (A4: BR-JR-06)

CMCMIS Phase 6 · Slice 1 · Manual Build Guide · Page 77 of 91

```
"submitted_by_user_id": 99999
}' | jq
# Expected response status: 400 (validator rejects unknown field – .strict() schema)
# Body: {"errors":{"submitted_by_user_id":"Unrecognized key(s) in object"}}
```

✓ Pass when

A 400 with an "Unrecognized key" message comes back. This proves the zod schema is .strict() and BR-JR-06 cannot be bypassed by the client.

Smoke #5 — Create draft → submit (A5: TnC gate)

```
bash
# Create the draft (legitimate body)
DRAFT=$(curl -s -b /tmp/normal.cookies -H "Content-Type: application/json" \
-X POST http://localhost:3000/api/v1/job-requests \
-d '{
  "job_category":"TME","job_type":"CALIBRATION",
  "equipment_name":"Multimeter Fluke 87V",
  "make":"Fluke","model_no":"87V","serial_no":"FL-001",
  "division_id":1,
  "complaint_description":"Annual calibration due. Last cal 12 months ago.",
  "priority":"MEDIUM"
}')
echo $DRAFT | jq
# Expected: {"data":{"id":N,"request_code":"JR-2026-XXXX","status":"DRAFT"}}

JR_ID=$(echo $DRAFT | jq -r '.data.id')

# Try to submit WITHOUT tnc_accepted: true
curl -i -b /tmp/normal.cookies -H "Content-Type: application/json" \
-X POST http://localhost:3000/api/v1/job-requests/$JR_ID/submit \
-d '{"tnc_accepted": false}'
# Expected: HTTP/1.1 400 Bad Request
# Body contains: tnc_accepted -> Invalid literal value, expected true

# Now submit correctly
curl -s -b /tmp/normal.cookies -H "Content-Type: application/json" \
-X POST http://localhost:3000/api/v1/job-requests/$JR_ID/submit \
-d '{"tnc_accepted": true, "tnc_version": "v1"}' | jq
# Expected: {"data":{"id":N,"request_code":"JR-2026-XXXX","status":"SUBMITTED"}}
```

Smoke #6 — Audit row written in same tx (A6)

```
bash
mysql -u cmcmis_dev -p cmcmis_dev_db -e "
  SELECT action, before_json, after_json, actor_user_id, created_at
  FROM audit_log
  WHERE entity = 'job_request' AND entity_id = $JR_ID
  ORDER BY id"
# Expected: two rows
# 1) action=CREATE_DRAFT before=NULL after={request_code,...,status:'DRAFT'}
# 2) action=SUBMIT before={status:'DRAFT'} after={status:'SUBMITTED',...}
```

✓ Pass when

Two audit rows exist with the same actor_user_id. If the SUBMIT audit row is missing but the status changed, the writeAudit call is outside the transaction (or absent) — fix the service layer.

Smoke #7 — Sort whitelist enforces safe ORDER BY (A7)

```
bash
# Try a bogus sort key – should be rejected
curl -i -b /tmp/normal.cookies "http://localhost:3000/api/v1/job-requests?sort=password"
# Expected: HTTP/1.1 400 Bad Request
# Body: {"errors":{"sort":"Invalid enum value..."}}
```

Smoke #8 — Pagination math (A8)

```
bash
curl -s -b /tmp/lic.cookies \
  "http://localhost:3000/api/v1/job-requests?page=1&page_size=10" | jq '.pagination'
# Expected: {
#   "page": 1, "page_size": 10,
#   "total": N, "total_pages": ceil(N/10)
# }

# Invalid page_size – must be in {10,25,50,100}
curl -i -b /tmp/lic.cookies \
  "http://localhost:3000/api/v1/job-requests?page_size=15"
# Expected: HTTP/1.1 400, errors.page_size
```

Smoke #9 — Filters preserved in URL (A9)

i Browser-side check

Open <http://localhost:5173/job-requests> in the browser.
 Type "fluke" in the search box, pick Status = Pending, change rows-per-page to 50.
 Hit Reload (Cmd/Ctrl+R) — every filter remains. The URL is `/job-requests?q=fluke&status=SUBMITTED&page_size=50`.
 Copy-paste that URL into another tab — same view loads.

Smoke #10 — EXPLAIN uses the new index (A10)

```
bash
mysql -u cmcmis_dev -p cmcmis_dev_db -e "
  EXPLAIN SELECT id, request_code FROM job_requests
  WHERE status = 'SUBMITTED'
  ORDER BY created_at DESC, id DESC
  LIMIT 25"
# Expected: key = idx_jr_list_default, rows = small number, Extra = "Using index"
```

Smoke #11 — p50 list latency under 50 ms (A11)

```
bash
# Quick-and-dirty latency check (use ab or wrk for serious numbers)
for i in $(seq 1 20); do
  curl -s -b /tmp/lic.cookies -o /dev/null \
    -w "%{time_total}\n" \
```

```
"http://localhost:3000/api/v1/job-requests?page=1&page_size=25"
done | sort -n | awk 'NR==10 { printf("p50: %.3fs\n", $1) }'
# Expected p50 < 0.050 (50 ms) on a warm DB
```

Smoke #12 — Job Cards list permission (A12)

```
bash
# Normal User cannot read job cards
curl -i -b /tmp/normal.cookies http://localhost:3000/api/v1/job-cards
# Expected: HTTP/1.1 403 Forbidden

# View-Only CAN read job cards
curl -s -b /tmp/vo.cookies http://localhost:3000/api/v1/job-cards | jq '.pagination'
# Expected: { "page":1, "page_size":25, "total":N, "total_pages":... }
```

✓ All 12 pass?

Slice 1 is functionally complete.

Tag the commit: git tag phase6-slice1-green.

Take a snapshot of audit_log for the test runs — useful in regression.

Move on to Slice 2: approve/reject UI for the Lab In-Charge.

Part V — What Changed (Diff Summary)

A precise enumeration of every file you added, every file you patched, and every database object that was created or modified. Use this as a code-review checklist.

Files Added (39)

Backend (21 new files)

#	Path	Lines (≈)
1	backend/db/discovery/0001_phase6_introspect.sql	40
2	backend/db/discovery/SCHEMA_PHASE6.md	70
3	backend/db/migrations/0010_phase6_columns.sql	60
4	backend/db/migrations/0011_phase6_indexes.sql	55
5	backend/db/migrations/0012_phase6_status_history.sql	30
6	backend/src/utlis/auditLog.js	40
7	backend/src/utlis/codeGenerator.js	55
8	backend/src/modules/jobRequests/jobRequests.validators.js	60
9	backend/src/modules/jobRequests/jobRequests.stateMachine.js	55
10	backend/src/modules/jobRequests/jobRequests.repo.js	130
11	backend/src/modules/jobRequests/jobRequests.service.js	100
12	backend/src/modules/jobRequests/jobRequests.controller.js	35
13	backend/src/modules/jobRequests/jobRequests.routes.js	30
14	backend/src/modules/jobCards/jobCards.validators.js	25
15	backend/src/modules/jobCards/jobCards.repo.js	70
16	backend/src/modules/jobCards/jobCards.service.js	20
17	backend/src/modules/jobCards/jobCards.controller.js	15
18	backend/src/modules/jobCards/jobCards.routes.js	20
19	backend/src/modules/lookups/lookups.repo.js	25
20	backend/src/modules/lookups/lookups.controller.js	25
21	backend/src/modules/lookups/lookups.routes.js	20

Frontend (18 new files)

#	Path	Lines (≈)
22	frontend/src/api/jobRequests.api.js	45
23	frontend/src/api/jobCards.api.js	18
24	frontend/src/api/lookups.api.js	40
25	frontend/src/schemas/jobRequestSchemas.js	40
26	frontend/src/schemas/jobCardSchemas.js	8
27	frontend/src/pages/jobRequests/components/StatusPill.jsx	30
28	frontend/src/pages/jobRequests/components/PriorityLabel.jsx	20
29	frontend/src/pages/jobRequests/components/PaginationBlock.jsx	70
30	frontend/src/pages/jobRequests/components/JobRequestFilters.jsx	60
31	frontend/src/pages/jobRequests/components/JobRequestTable.jsx	70
32	frontend/src/pages/jobRequests/JobRequestList.jsx	85
33	frontend/src/pages/jobRequests/components/form/tncContent.js	15
34	frontend/src/pages/jobRequests/components/form/Section1JobType.jsx	35
35	frontend/src/pages/jobRequests/components/form/Section2Equipment.jsx	70
36	frontend/src/pages/jobRequests/components/form/Section3Accessories.jsx	55
37	frontend/src/pages/jobRequests/components/form/Section4SubmittedBy.jsx	110
38	frontend/src/pages/jobRequests/components/form/Section5TermsAndConditions.jsx	45
39	frontend/src/pages/jobRequests/JobRequestNew.jsx	95
40	frontend/src/pages/jobCards/components/JobCardTable.jsx	55
41	frontend/src/pages/jobCards/JobCardList.jsx	85

Files Patched (6)

Path	What changed
backend/src/middleware/rowLevelScope.js	REPLACE the Phase 4 stub with the real implementation. Sets req.scope.onlyOwnerId for /api/v1/job-requests (Normal Users) and req.scope.engineerId for /api/v1/job-cards (Lab Engineers).
backend/src/modules/users/users.controller.js	ADD getProfile() — returns full user including division join + lab/room phone for Section 4 auto-fill.
backend/src/modules/users/users.routes.js	ADD route GET /users/me/profile alongside the existing GET /users/me.
backend/src/server.js	ADD three app.use lines: /api/v1/job-requests, /api/v1/job-cards, /api/v1/lookups (plus two imports).
frontend/src/lib/permissions.js	ADD two ALL_NAV_ITEMS entries: Job Requests (gates on job_request:read-own) and Job Cards (gates on job_card:read-list).
frontend/src/App.jsx	ADD three <Route> blocks wrapped in ProtectedRoute for /job-requests, /job-requests/new, /job-cards.
frontend/tailwind.config.js	EXTEND theme.extend.colors.status with eight bg/fg pairs (draft, pending, approved, in_progress, completed, verified_closed, rejected, reopened).

DB Objects Created (15)

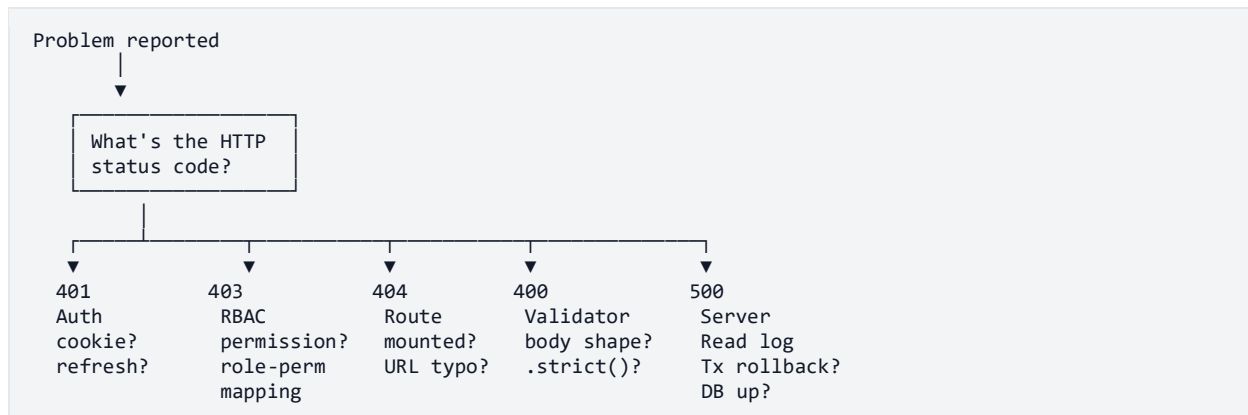
Object	Type	Purpose
job_requests.request_code	column (added if missing)	JR-2026-NNNN identity
job_requests.priority	column (added if missing)	LOW MEDIUM HIGH
job_requests.accessories	column (added if missing)	JSON array
job_requests.tnc_accepted_at, tnc_version	columns (added if missing)	TnC double-lock evidence
job_requests.submitted_at	column (added if missing)	Wall-clock of state change DRAFT→SUBMITTED
job_requests.division_code	column (added if missing)	denormalised for fast list display

Object	Type	Purpose
job_requests.submitted_by_name/emp_id/designation/email	columns (added if missing)	BR-JR-06 snapshot fields
idx_jr_list_default	BTREE index	covering: (status, created_at DESC, id)
idx_jr_owner	BTREE index	covering: (submitted_by_user_id, created_at DESC)
idx_jr_division	BTREE index	covering: (division_id, created_at DESC)
idx_jr_priority_status	BTREE index	covering: (priority, status, created_at DESC)
idx_jr_code	BTREE index	lookup by request_code
ft_jr_search	FULLTEXT index	search across code+name+serial+submitter
idx_jc_list_default, idx_jc_engineer, idx_jc_jr	BTREE indexes	job_cards list + engineer filter + JR join
job_request_status_history, job_card_status_history	tables (IF NOT EXISTS)	audit trail of state transitions
code_counters	table	sequence backing JR-/JC-code generator

Part VI — Troubleshooting

When something fails, do not guess. Walk the decision tree below first. The vast majority of Phase 6 bugs fall into one of seven buckets.

How to think when things break



Bucket 1 · 401 Unauthorized

i Symptom

Every API call returns 401, even after login.

- Check the browser DevTools → Network → Headers: is Cookie sent on the request? If not, the FE axios instance is missing withCredentials: true.
- Check the Set-Cookie on the login response: cookie path must be "/" and SameSite must allow your dev domain.
- Restart the backend after editing .env — JWT secrets are loaded once at boot.
- Token expired? Phase 4 sets the access token at 15 min. The refresh interceptor on the axios client should renew it automatically — if missing, sessions die after 15 min.

Bucket 2 · 403 Forbidden

i Symptom

POST /job-requests returns 403 for a user who should be able to create.

- `curl http://localhost:3000/api/v1/users/me -b cookies` — look at the permissions array in the response.
- Is `job_request:create` present? If not, the `role_permissions` seed is missing rows. Re-run the Phase 3 seed.
- Permission name typo? Check the routes file — `authorize('job_request:create')` must match the exact string in the `role_permissions` table.

Bucket 3 · 404 Not Found

i Symptom

POST /api/v1/job-requests returns 404.

- You forgot Step 6: server.js does not have the `app.use('/api/v1/job-requests', ...)` line.
- Path typo — singular "job-request" instead of plural? Express is case- and dash-sensitive.
- Restart the backend after editing server.js. Vite-style HMR does not apply to Node.

Bucket 4 · 400 Bad Request

i Symptom

POST /job-requests with what looks like a valid body returns 400.

- Read the response body — the validator returns { errors: { field: "message" } }.
- Unrecognized key error → you are sending a field the zod schema does not declare. Because the schema is .strict(), unknown fields are rejected. Remove it.
- Type errors → zod expects a number but you sent a string ("1" vs 1). Use Number() before sending, or use z.coerce.number() on the BE side for query params (already done on listQuerySchema).
- tnc_accepted invalid literal → you sent false or omitted it. The form must reach (6/6) before submit.

Bucket 5 · 500 Internal Server Error

! Stop. Read the pino log line.

The pino log line is verbose JSON with a stack trace. Find the file + line.

Most common cause in Phase 6: the repo SELECT references a column that does not exist (real column name differs from canonical).

Solution: open SCHEMA_PHASE6.md, find the real name, alias it in the repo (SELECT real_name AS canonical_name).

- ER_NO_SUCH_TABLE → migration was not applied to the right database. Confirm DB_NAME in .env and re-run migrations.
- ER_BAD_FIELD_ERROR → column does not exist. Either run 0010_phase6_columns.sql or alias in repo.
- Cannot read property of undefined → likely req.user is missing because a route forgot to add authenticate before its handler. Check the routes file.

Bucket 6 · Pagination shows wrong total

i Symptom

List shows "Showing 25 of 25" but you know there are 100 rows.

- The COUNT(*) query is being filtered differently than the data query. Check repo.findList — the where + args arrays must be identical for both.
- rowLevelScope is being applied only to the data query, not the count. The fix is in the repo: build whereSql once and use it for both.

Bucket 7 · Form submits but no row appears

i Symptom

POST returns 201 but the list does not refresh.

- `useCreateDraft` / `useSubmitRequest` must call `queryClient.invalidateQueries({ queryKey: ['job-requests'] })` in `onSuccess`. Without this, `react-query` keeps the cached list.
- After invalidation, the list refetches in the background. Until it does, the new row is invisible — wait one second or hard-refresh.
- If the row really did not save, check the `audit_log` table. If two rows exist (`CREATE_DRAFT` + `SUBMIT`), the data is in DB and the bug is in the FE invalidation.

✓ Golden rule

Log first. Schema second. Permissions third. Code last. Most "code bugs" turn out to be one of the first three.

Appendix A — Full Permission Reference

All permission codes that touch Phase 6, with the exact roles that own each one. Source: FINAL-DESC §6.5.

Code	Normal	View-Only	Lab Eng.	Lab In-Charge	Super Admin	Notes
dashboard:view	✓	✓	✓	✓	✓	Phase 4
equipment:read-list	✓	✓	✓	✓	✓	Phase 5
equipment:read-detail	✓	✓	✓	✓	✓	Phase 5
equipment:create	X	X	X	✓	✓	Phase 5
job_request:create	✓	X	✓	✓	✓	POST /job-requests
job_request:read-own	✓	✓	✓	✓	✓	enables nav item
job_request:read-all	X	✓	✓	✓	✓	bypasses rowLevelScope
job_request:approve	X	X	X	✓	✓	Slice 2
job_request:reject	X	X	X	✓	✓	Slice 2
job_request:assign-engineer	X	X	X	✓	✓	Slice 2
job_card:read-list	X	✓	✓	✓	✓	enables nav item
job_card:read-detail	X	✓	✓	✓	✓	Slice 2
audit_log:read	X	X	X	X	✓	Phase 7
user:read-list	X	X	X	X	✓	Phase 7

Appendix B — Status Badge Color Map

Light, semantically meaningful, accessibility-checked (WCAG AA on light backgrounds).

Status	Display label	Tailwind bg	Tailwind fg	Hex bg	Hex fg
DRAFT	Draft	bg-slate-100	text-slate-700	#F1F5F9	#334155
SUBMITTED	Pending	bg-amber-100	text-amber-700	#FEF3C7	#92400E
ASSIGNED	Approved	bg-violet-100	text-violet-700	#EDE9FE	#5B21B6
IN_PROGRESS	In Progress	bg-blue-100	text-blue-700	#DBEAFE	#1E40AF

Status	Display label	Tailwind bg	Tailwind fg	Hex bg	Hex fg
COMPLETED	Completed	bg-green-100	text-green-700	#DCFCE7	#166534
VERIFIED_CLOSED	Verified	bg-emerald-100	text-emerald-700	#D1FAE5	#065F46
REJECTED	Rejected	bg-red-100	text-red-700	#FEE2E2	#991B1B
REOPENED	Reopened	bg-orange-100	text-orange-700	#FFEDD5	#9A3412
CANCELLED	Cancelled	bg-slate-100	text-slate-500	#F1F5F9	#64748B

i End of Phase 6 · Slice 1 manual build guide

You now hold every line of code, every migration, every test, and every troubleshooting note required to ship Slice 1 by hand.

Next: Slice 2 — Job Request detail view, approve/reject UI for the Lab In-Charge, and the auto-create-on-approve hook that lands the first row in `job_cards`.

Tag this build: `git tag phase6-slice1-green` and commit the `SCHEMA_PHASE6.md` you wrote in Step 0 — future-you will thank present-you.